

[University / School name — placeholder]

[Department — placeholder]

End-of-Studies Project Report

Submitted in partial fulfilment of the requirements for the degree of [Degree title — placeholder]

Inkwell.ai

An AI-powered writing marketplace connecting independent writers and magazine publishers

Carried out by: **Oussama Bejaoui**

Academic supervisor: [Name — placeholder]

Professional supervisor: [Name — placeholder]

Host organisation: [Organisation — placeholder]

Project period: **1 February 2026 – 31 July 2026**

Academic year **2025–2026**

Acknowledgements

[Placeholder — to be written by the author: thanks to the academic supervisor, the professional supervisor and the host organisation, the members of the jury, and family and friends.]

Abstract

Independent writers have tools to publish but few ways to be paid for quality, and magazines that want to commission them have no objective way to judge a writer before buying. This report presents **Inkwell.ai**, a two-sided web platform that addresses both sides. Writers draft in a rich-text editor beside an AI assistant grounded in their own published work and in reference documents they upload; it answers in a panel or writes directly into the document, cites its sources by page, and accepts spoken prompts. Every published article feeds an event-based analytics pipeline whose audience, content and quality signals, together with AI-generated portfolio insights, form an evaluation report that magazines use to decide. Magazines subscribe, receive monthly credits, preview an article for 10% of its price and buy the remainder for exclusive republication rights, every credit movement being recorded in a ledger whose invariants are tested after each operation.

The platform was built over nine two-week Scrum sprints grouped into four releases, with a Next.js 16 frontend, a NestJS 11 backend and worker, PostgreSQL 16 with pgvector, Redis and BullMQ, MinIO object storage and nginx, all containerised with Docker Compose, and with Groq and Gemini models orchestrated through the Vercel AI SDK. The backend suite counts 792 passing tests in 51 suites.

Keywords: writing marketplace, AI-assisted writing, retrieval-augmented generation, pgvector, analytics, article licensing, Scrum, NestJS, Next.js.

Résumé

Les rédacteurs indépendants disposent d'outils pour publier, mais de peu de moyens d'être rémunérés pour la qualité de leur travail, et les magazines qui souhaitent leur confier des articles n'ont aucun moyen objectif d'évaluer un rédacteur avant d'acheter. Ce rapport présente **Inkwell.ai**, une plateforme web biface qui répond aux deux besoins. Les rédacteurs écrivent dans un éditeur de texte enrichi, aux côtés d'un assistant d'intelligence artificielle ancré dans leurs propres articles publiés et dans les documents de référence qu'ils déposent ; l'assistant répond dans un panneau ou écrit directement dans le document, cite ses sources à la page près et accepte des consignes dictées à la voix. Chaque article publié alimente une chaîne d'analyse fondée sur des événements, dont les indicateurs d'audience, de contenu et de qualité, complétés par une synthèse générée par l'IA, constituent un rapport d'évaluation sur lequel les magazines s'appuient pour décider. Les magazines s'abonnent, reçoivent des crédits mensuels, débloquent un aperçu pour 10 % du prix et achètent le reste pour obtenir des droits de republication exclusifs ; chaque mouvement de crédits est inscrit dans un grand livre dont les invariants sont vérifiés après chaque opération.

La plateforme a été réalisée en neuf sprints Scrum de deux semaines, regroupés en quatre releases, avec un frontend Next.js 16, un backend et un worker NestJS 11, PostgreSQL 16 avec pgvector, Redis et BullMQ, le stockage objet MinIO et nginx, le tout conteneurisé avec Docker Compose, les modèles Groq et Gemini étant orchestrés par le Vercel AI SDK. La suite de tests du backend compte 792 tests réussis répartis en 51 suites.

Mots-clés : place de marché d'écriture, rédaction assistée par l'IA, génération augmentée par la recherche, pgvector, analytique, licence d'articles, Scrum, NestJS, Next.js.

List of acronyms

Acronym	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Deployment
CRUD	Create, Read, Update, Delete
DTO	Data Transfer Object
FR / NFR	Functional / Non-Functional Requirement
GHCR	GitHub Container Registry
HNSW	Hierarchical Navigable Small World (approximate nearest-neighbour index)
JWT	JSON Web Token
LLM	Large Language Model
MoSCoW	Must, Should, Could, Won't (prioritisation)
ORM	Object-Relational Mapping
PFE	Projet de Fin d'Études (end-of-studies project)
RAG	Retrieval-Augmented Generation
RRF	Reciprocal Rank Fusion
S3	Simple Storage Service (object storage protocol)
SSE	Server-Sent Events
TLS	Transport Layer Security
TTS	Text-to-Speech
UML	Unified Modeling Language
US	User Story

Table of contents

- General introduction
- Chapter 1 — Project context and methodology
 - Introduction
 - 1.1 Host organisation
 - 1.2 Project context
 - 1.3 Study of existing solutions
 - 1.4 Problem statement
 - 1.5 Proposed solution
 - 1.6 Development methodology
 - Conclusion
- Chapter 2 — Requirements analysis and specification
 - Introduction
 - 2.1 Identification of the actors
 - 2.2 Functional requirements
 - 2.3 Non-functional requirements
 - 2.4 Global use case diagram
 - 2.5 Global class diagram
 - 2.6 Product backlog
 - 2.7 Release plan
 - 2.8 Schedule
 - Conclusion
- Chapter 3 — Working environment and architecture
 - Introduction
 - 3.1 Working environment
 - 3.2 Technological choices
 - 3.3 Logical architecture
 - 3.4 Physical architecture
 - 3.5 Continuous integration and delivery
 - Conclusion
- Chapter 4 — Release 1: functional foundation
 - Introduction
 - 4.1 Sprint 1 — Schema, authentication and article core
 - 4.2 Sprint 2 — Editor, AI assistance and event capture
 - Conclusion
- Chapter 5 — Release 2: social interactions, analytics and AI
 - Introduction
 - 5.1 Sprint 3 — Social interactions, notifications and analytics
 - 5.2 Sprint 4 — Retrieval, portfolio insights and hybrid search

- Conclusion
- Chapter 6 — Release 3: marketplace, testing and deployment
 - Introduction
 - 6.1 Sprint 5 — Marketplace, subscriptions, premium and moderation
 - 6.2 Sprint 6 — Tests, demonstration and deployment
 - Conclusion
- Chapter 7 — Release 4: social depth, the reworked assistant and documents
 - Introduction
 - 7.1 Sprint 7 — Comment likes, lists, block, save and share
 - 7.2 Sprint 8 — Live cards, the assistant that writes, and voice
 - 7.3 Sprint 9 — Document library and citations
 - Conclusion
- General conclusion and perspectives
 - Limitations
 - Perspectives
- References

List of figures

- Figure 1.2 – Classic and agile approaches compared
- Figure 1.1 – The Scrum framework, as practised in this project
- Figure 2.1 – Global use case diagram
- Figure 2.2 – Class diagram: core domain
- Figure 2.3 – Class diagram: AI, marketplace and analytics
- Figure 2.4 – Gantt chart of the project
- Figure 3.1 – Logical frontend architecture
- Figure 3.2 – Logical backend architecture
- Figure 3.3 – Physical architecture
- Figure 3.4 – CI/CD pipeline
- Figure 4.1 – Use case diagram, Sprint 1
- Figure 4.2 – Class diagram, Sprint 1
- Figure 4.3 – Sequence diagram: sign in and silent refresh
- Figure 4.4 – The public feed, as seen by a visitor
- Figure 4.5 – Sign-in, with Google sign-in
- Figure 4.6 – Registering a personal account
- Figure 4.7 – Registering a magazine account
- Figure 4.8 – A free article, readable without an account
- Figure 4.9 – Profile settings
- Figure 4.10 – Use case diagram, Sprint 2
- Figure 4.11 – Class diagram, Sprint 2
- Figure 4.12 – Sequence diagram: inline AI action
- Figure 4.13 – The rich-text editor with the assistant panel
- Figure 4.14 – The publish dialog: placement, visibility, tags and excerpt
- Figure 4.15 – The AI assistant panel
- Figure 4.16 – An answer from the assistant
- Figure 4.17 – The inline actions on a selection
- Figure 4.18 – An inline suggestion, ready to replace the selection
- Figure 5.1 – Use case diagram, Sprint 3
- Figure 5.2 – Class diagram, Sprint 3
- Figure 5.3 – Sequence diagram: analytics aggregation
- Figure 5.4 – Sequence diagram: live notification delivery
- Figure 5.5 – The feed of a signed-in reader, with live card controls
- Figure 5.6 – Notifications, filtered and grouped by day
- Figure 5.7 – The writer’s dashboard
- Figure 5.8 – “My articles”: every article, sortable by performance
- Figure 5.9 – The writer’s analytics: views over the last 30 days
- Figure 5.10 – A writer’s public profile
- Figure 5.11 – A magazine browsing eligible writers
- Figure 5.12 – Use case diagram, Sprint 4

- Figure 5.13 – Class diagram, Sprint 4
- Figure 5.14 – Sequence diagram: the publishing pipeline
- Figure 5.15 – Sequence diagram: AI chat with retrieval
- Figure 5.16 – Sequence diagram: hybrid search with reciprocal rank fusion
- Figure 5.17 – Search results
- Figure 5.18 – A writer evaluation report with portfolio insights
- Figure 6.1 – Use case diagram, Sprint 5
- Figure 6.2 – Class diagram, Sprint 5
- Figure 6.3 – Sequence diagram: preview, purchase and publication
- Figure 6.4 – Activity diagram: the marketplace eligibility gate
- Figure 6.5 – Sequence diagram: reporting and moderation
- Figure 6.6 – Activity diagram: the article access decision
- Figure 6.7 – State diagram: the article lifecycle
- Figure 6.8 – A premium article behind the paywall
- Figure 6.9 – The plans: free and premium
- Figure 6.10 – A magazine’s subscription and credits
- Figure 6.11 – The marketplace, with preview and purchase prices
- Figure 6.12 – A listing before preview
- Figure 6.13 – Confirming a preview unlock: price and balance after
- Figure 6.14 – The magazine’s library: unpublished and published
- Figure 6.15 – Publishing a purchased article under the magazine’s masthead
- Figure 6.16 – A licensed article naming both the writer and the magazine
- Figure 6.17 – A magazine’s public profile and its published articles
- Figure 6.18 – A writer’s earnings, itemised by preview and purchase
- Figure 6.19 – A sold article, as its author sees it
- Figure 6.20 – Administration: manual eligibility grant
- Figure 6.21 – Administration: user management
- Figure 6.22 – Administration: the report queue
- Figure 6.23 – Test strategy
- Figure 7.1 – Use case diagram, Sprint 7
- Figure 7.2 – Class diagram, Sprint 7
- Figure 7.3 – The Saved shelf
- Figure 7.4 – Blocked accounts, in settings
- Figure 7.5 – The following feed
- Figure 7.6 – A follower list, searched
- Figure 7.7 – Use case diagram, Sprint 8
- Figure 7.8 – Sequence diagram: one assistant turn
- Figure 7.9 – Sequence diagram: writing into the document
- Figure 7.10 – Sequence diagram: the voice round trip
- Figure 7.11 – The Reposted tab of a profile
- Figure 7.12 – A write awaiting Keep or Discard
- Figure 7.13 – The live step on the collapsed panel
- Figure 7.14 – Recording a prompt

- Figure 7.15 – Use case diagram, Sprint 9
- Figure 7.16 – Class diagram, Sprint 9
- Figure 7.17 – Sequence diagram: document upload and ingestion
- Figure 7.18 – The document library, one document refused with its reason
- Figure 7.19 – Attaching a document from the Sources tab
- Figure 7.20 – A reply citing its source as [Title, p. N]

List of tables

- Table 1.1 – Comparison of existing solutions
- Table 2.1 – Actors of the system
- Table 2.2 – Functional requirements of the visitor
- Table 2.3 – Functional requirements of the readers
- Table 2.4 – Functional requirements of the writers
- Table 2.5 – Functional requirements of the magazines
- Table 2.6 – Functional requirements of the administrator
- Table 2.7 – Cross-cutting functional requirements
- Table 2.8 – Non-functional requirements
- Table 2.9 – Product backlog by epic
- Table 2.10 – Release plan
- Table 3.1 – Software tools
- Table 3.2 – Frontend technologies
- Table 3.3 – Backend technologies
- Table 3.4 – AI models and their use
- Table 4.1 – Sprint 1 backlog
- Table 4.2 – Textual description of “Publish an article”
- Table 4.3 – Sprint 2 backlog
- Table 5.1 – Sprint 3 backlog
- Table 5.2 – Main metrics of the evaluation report
- Table 5.3 – Sprint 4 backlog
- Table 6.1 – Sprint 5 backlog
- Table 6.2 – Marketplace rules
- Table 6.3 – Sprint 6 backlog
- Table 6.4 – Test results
- Table 7.1 – Sprint 7 backlog
- Table 7.2 – Sprint 8 backlog
- Table 7.3 – Sprint 9 backlog

General introduction

Writing has never been easier to publish and never harder to be paid for. Blogging platforms and newsletter services let anyone reach readers in minutes, and large language models now help with drafting, rephrasing and editing. Yet an independent writer who produces consistently good work still has few routes to income beyond advertising or reader subscriptions, and the publications that would happily pay for that work struggle to find it — and, once they find it, to judge it on anything firmer than a few samples and intuition.

This end-of-studies project starts from that double gap. On one side, writers need an assistant that helps them write *in their own voice*, not a generic text generator; on the other, magazines need objective signals — who reads a writer, how far, how often they come back — before they commit money. Between the two, a market is missing: a place where a magazine can browse, evaluate and license an existing article directly from its author, with clear terms and a trustworthy record of who paid what.

Inkwell.ai is the platform built to fill that gap. It combines three pillars:

1. **AI-collaborative writing** — an assistant integrated into the editor as chat, inline actions and voice input, grounded in the writer’s published corpus and in reference documents the writer uploads, able to write directly into the draft;
2. **decision-support analytics** — every published article generates audience, content and quality signals that feed dashboards for the writer and an evaluation report for magazines, completed by AI-generated portfolio insights;
3. **an article licensing marketplace** — magazines are a distinct account type that subscribes, receives monthly credits, previews an article for a tenth of its price and buys the rest for exclusive republication rights.

The work was carried out over six months, from February to July 2026, following the Scrum framework in nine fixed two-week sprints grouped into four releases. This report follows that structure:

- **Chapter 1** presents the context of the project, studies existing solutions, states the problem and justifies the choice of Scrum as the development method.
- **Chapter 2** analyses and specifies the requirements: actors, functional and non-functional requirements, the global use case and class diagrams, the product backlog and the release plan.
- **Chapter 3** describes the working environment, the technological choices and the logical and physical architecture of the system.
- **Chapters 4 to 7** present the four releases sprint by sprint — the sprint backlog, the analysis and design (use case, class and sequence diagrams) and the realisation — from the functional foundation to the marketplace, testing and deployment, and finally to the reworked assistant and the document library.

A general conclusion summarises the results and opens perspectives for the platform.

Chapter 1 — Project context and methodology

Introduction

This chapter situates the project. It presents the host organisation and the context in which Inkwell.ai was conceived, studies the existing solutions on both sides of the market, states the problem the project addresses and the solution it proposes, and closes with the development methodology adopted and the reasons for choosing it.

1.1 Host organisation

[Placeholder — presentation of the host organisation: identity, field of activity, organisation chart and the team the project was carried out in. To be completed by the author.]

1.2 Project context

The project was proposed as an end-of-studies engineering project with an ambitious, product-shaped scope: not a prototype of one feature, but a complete two-sided platform with real accounts, real data flows, an AI layer and money-like transactions. Three observations motivated it.

- **Generative AI has changed how text is produced**, but most writing assistants are generic: they write *for* the user rather than *like* the user, and they know nothing of what the writer has already published.
- **Engagement data is collected everywhere and used for very little**. Platforms measure views and reading time to rank content for readers, but none exposes those signals as evidence to a buyer evaluating a writer.
- **Content licensing between independent writers and publications is informal**. It happens by e-mail, on personal relationships, with no shared record of terms or payments.

1.3 Study of existing solutions

1.3.1 Publishing and newsletter platforms

Medium is an open publishing platform where readers subscribe to the platform and writers in its partner programme are paid according to member reading time. It offers good reading surfaces and audience statistics, but no AI assistance grounded in the writer's corpus, and no way for a publication to license an article.

Substack lets writers run paid newsletters: readers subscribe to a writer directly. It has made independent writing a viable business for some, but its economic model is the reader subscription; there is no marketplace in which a publication buys an existing piece.

1.3.2 Freelance content marketplaces

Content marketplaces such as **Contently** or general freelance platforms connect brands with writers for *commissioned* work. Evaluation rests on portfolios, ratings and interviews; the work does not yet exist when it is bought, and the platform holds no engagement data about the writer's published pieces.

1.3.3 AI writing assistants

Tools such as **Grammarly** or **Notion AI** assist with grammar, tone and generation inside a document. They are powerful editors, but they are not grounded in the writer's own body of work, they do not cite the sources a passage came from, and they are disconnected from any publishing or licensing workflow.

1.3.4 Comparative synthesis

Table 1.1 compares these families of solutions with the criteria the project targets.

Table 1.1 – Comparison of existing solutions

Criterion	Medium	Substack	Content marketplaces	AI writing assistants	Inkwell.ai
Publishing and public reading	✓	✓	—	—	✓
AI assistance in the editor	—	—	—	✓	✓
Assistant grounded in the writer's own work	—	—	—	—	✓
Answers citing uploaded documents by page	—	—	—	partial	✓
Engagement analytics for the writer	✓	✓	—	—	✓
Analytics exposed to a buyer as evidence	—	—	—	—	✓
Licensing of existing articles to publications	—	—	—	—	✓
Traceable payments (ledger)	partial	✓	✓	—	✓

1.4 Problem statement

The study shows that each existing family covers part of the need and that the parts do not meet. The problem can be stated on three levels:

- **Supply side.** Writers lack an assistant that helps them write better *in their own voice*, grounded in what they have already published and in the material they are working from, and they lack a direct path to be paid for quality.
- **Demand side.** Magazines lack objective, comparable signals about a writer's audience, consistency and quality, and must evaluate by hand.
- **Market.** There is no self-service place where a publication can discover, evaluate, preview and license an existing article, with the money recorded in a way both parties can trust.

1.5 Proposed solution

Inkwell.ai answers these three levels with one platform:

- an **editor with an AI assistant** that retrieves passages from the writer's own published articles and attached reference documents, decides per request whether to answer in a panel or write into the document, streams what it writes into a protected range the writer keeps or discards, cites passages by page, and accepts spoken prompts;
- an **analytics pipeline** that turns reading events into per-article and per-writer metrics, presented to the writer for self-improvement and to magazines as an evaluation report with AI portfolio insights;
- a **marketplace** gated by an eligibility threshold, where magazines with an active subscription preview an article for 10% of its price, buy the remaining 90% for exclusive rights, and publish it under their own masthead — every credit movement being written to a ledger inside one database transaction.

1.6 Development methodology

1.6.1 Sequential and agile approaches

A **sequential (waterfall) process** runs requirements, design, implementation, testing and delivery once each, in order; working software exists only at the end. An **agile process** builds the product in short iterations, each delivering a working increment that is reviewed before the next is planned. Figure 1.2 contrasts the two.

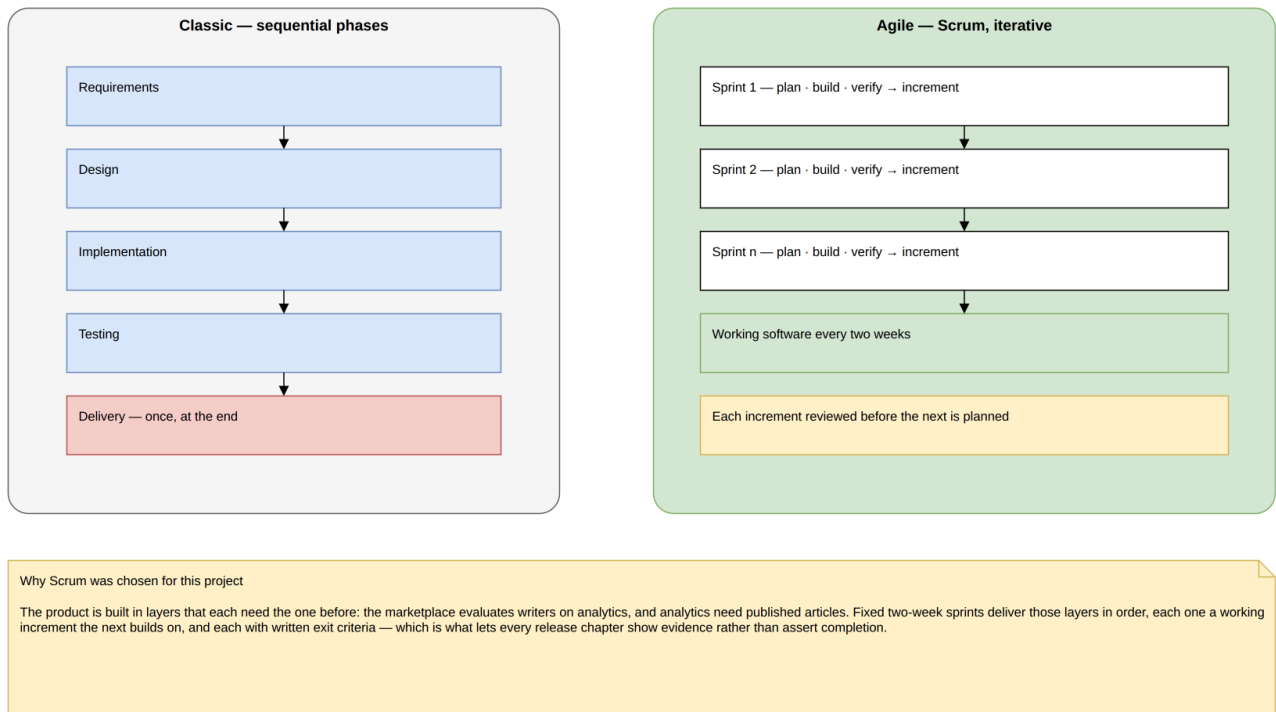


Figure 1.2 – Classic and agile approaches compared

1.6.2 Choice of Scrum

Inkwell.ai is built in layers that each depend on the previous one: the marketplace evaluates writers on analytics, analytics need published articles, and articles need accounts and an editor. An iterative method fits that layering: each sprint delivers a working layer that the next one builds on and that can be demonstrated on its own. Among agile frameworks, **Scrum** was chosen for its fixed-length sprints, its explicit artefacts — product backlog, sprint backlog, increment — and its events, which give an individual project the regular checkpoints a supervisor can follow.

1.6.3 Scrum as practised in this project

Figure 1.1 shows the framework as it was applied.

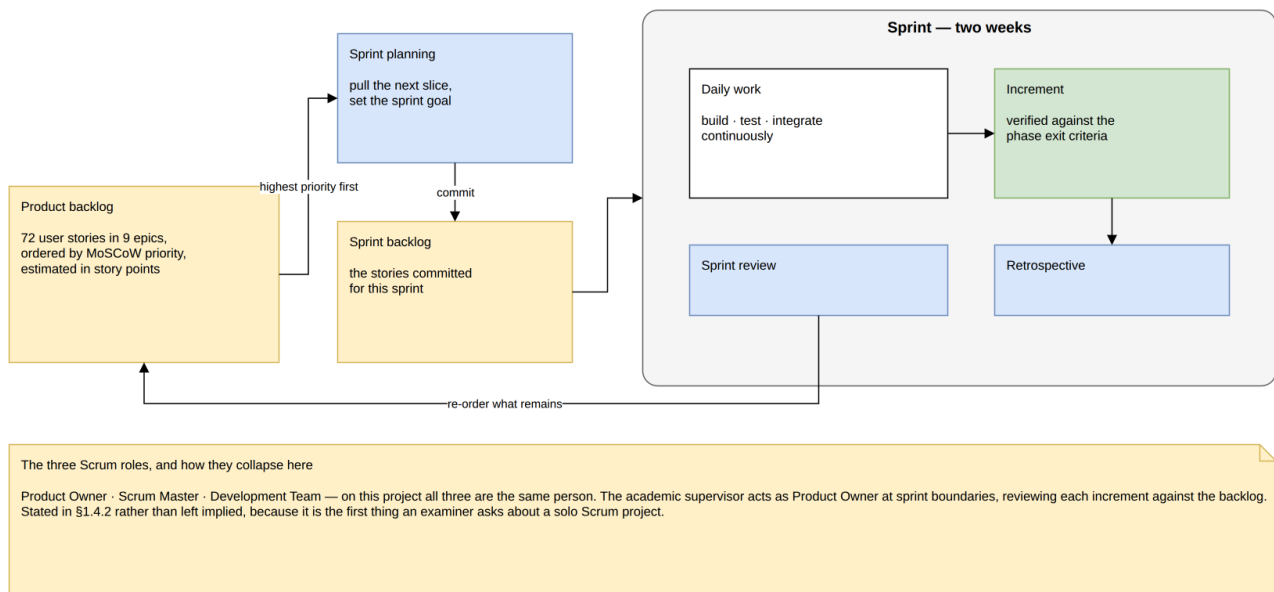


Figure 1.1 – The Scrum framework, as practised in this project

- **Roles.** The project is carried out by one person, who therefore holds the roles of Development Team and Scrum Master. The academic supervisor acts as Product Owner at sprint boundaries, reviewing each increment against the backlog.
- **Artefacts.** The product backlog holds 72 user stories in 9 epics, prioritised with MoSCoW and estimated in story points (Chapter 2). Each sprint has its own backlog and ends with a working increment.
- **Events.** Each two-week sprint starts with planning — pulling the highest-priority stories and setting a sprint goal — and ends with a review of the increment and a retrospective. Each phase of work carries written exit criteria, which is what lets every release chapter show evidence rather than assert completion.

1.6.4 Modelling language

Analysis and design use **UML 2**: use case diagrams for the functional scope of each sprint, class diagrams for the data model, sequence diagrams for the main interactions, and activity and state diagrams where a decision or a lifecycle is the subject. The diagrams are kept as text sources (PlantUML) or generated (draw.io) in the project repository, so they are versioned beside the code they describe.

Conclusion

This chapter presented the context of the project, compared the existing solutions and showed that none combines AI-assisted writing grounded in the writer’s own work, buyer-facing analytics and a licensing market. It stated the resulting problem and the proposed solution, and justified the use of Scrum. The next chapter analyses and specifies the requirements of the platform.

Chapter 2 — Requirements analysis and specification

Introduction

This chapter specifies what the platform must do. It identifies the actors, states the functional requirements per actor and the non-functional requirements, presents the global use case diagram and the global class diagrams, and then organises the work as a product backlog of user stories distributed over a release plan and a schedule.

2.1 Identification of the actors

The platform has eight actors. They are not independent: they generalise one another, mirroring the three independent properties every account carries in the data model — its **account type** (personal or magazine), its **role** (reader, writer or administrator) and its **plan** (free or premium) — plus the writer’s marketplace eligibility. Table 2.1 lists them.

Table 2.1 – Actors of the system

#	Actor	Data-model representation	Definition
A1	Visitor	no account	Unauthenticated. Sees the public feed and free article bodies; cannot interact.
A2	Free reader	personal account, free plan	Reads free public articles, interacts socially, has no AI allowance.
A3	Premium reader	personal account, premium plan	Everything A2 has, plus premium articles and the daily AI allowance.
A4	Writer	role writer	A2/A3 plus drafting, publishing and editing their own articles.
A5	Eligible writer	marketplace-eligible writer	A4 plus the right to publish premium articles and to list on the marketplace.
A6	Magazine (unsubscribed)	magazine account, no active subscription	Registered magazine stopped at the subscription wall.
A7	Magazine (subscribed)	magazine account, active subscription	Browses writers, reads evaluation reports, spends credits to preview and purchase.

#	Actor	Data-model representation	Definition
A8	Administrator	role admin	Moderation queue, user management, manual eligibility grants.

Three generalisations hold: the premium reader generalises the free reader, the eligible writer generalises the writer, and the subscribed magazine generalises the unsubscribed one. Role and plan are deliberately independent: a writer may stay on the free plan and write without AI assistance. The administrator role cannot be obtained through the interface; it is granted directly in the database.

2.2 Functional requirements

The functional requirements are stated per actor, each with a stable identifier, a MoSCoW priority and the sprint that delivers it. Identifiers are stable rather than sequential: a requirement added later keeps the next free number and is listed beside the feature it belongs to.

2.2.1 Visitor (A1)

Table 2.2 – Functional requirements of the visitor

ID	Requirement	Priority	Sprint
FR-01	Browse the public feed, paginated, with title, excerpt, thumbnail and tags	Must	1
FR-69	Continue any article list as the reader scrolls, with a “Load more” control that remains available and a failed page that neither discards loaded rows nor retries itself	Must	1
FR-02	Read the full body of a free public article without an account	Must	1
FR-03	Be blocked from premium and marketplace articles, with an explicit prompt to sign up	Must	1
FR-71	Not see a marketplace listing on the writer’s public profile at all — the exceptions being the article’s own author and an administrator	Must	1

ID	Requirement	Priority	Sprint
FR-75	Read a magazine's published articles on its public profile, paginated, without an account	Must	1
FR-76	See both the writer and the publishing magazine named on a licensed article, each linking to its own profile	Must	1
FR-77	Preview a writer or magazine from any byline by hovering it — name, bio, counts and a follow control for a writer; logo, description and founding date for a magazine — without leaving the page	Should	8
FR-04	Register as a personal account, or as a magazine account	Must	1
FR-05	Authenticate by email/password or Google OAuth	Must	1
FR-06	Search articles and writers, and browse by tag	Should	4

Free public articles are deliberately readable without an account: gating them would contradict the public feed and its search-engine visibility.

2.2.2 Free and premium reader (A2, A3)

Table 2.3 – Functional requirements of the readers

ID	Requirement	Priority	Sprint
FR-07	Manage a profile — display name, bio, avatar upload	Must	1
FR-08	Like an article, and un-like it	Must	3
FR-09	Comment on an article, and reply to a comment (threaded)	Must	3
FR-10	Delete one's own comment	Must	3
FR-60	Like a comment or a reply, and un-like it	Must	7

ID	Requirement	Priority	Sprint
FR-66	Like, repost and comment on an article from a feed card, and see whether one has already liked or reposted it	Must	8
FR-68	See a feed card's most recent comments from the last hour, up to three, without opening the article	Could	8
FR-11	Repost an article	Could	3
FR-70	Be refused when liking, commenting on or reposting a marketplace listing, while saving one stays available	Must	3
FR-67	Read any account's reposts from the Reposted tab on its profile, without an account	Should	8
FR-12	Follow and unfollow a writer	Should	3
FR-61	View any profile's followers and following lists, and follow or unfollow from them	Must	7
FR-62	Block an account, and review and lift one's blocks from settings	Must	7
FR-63	Save an article from the feed card or from the article page, and unsave it	Must	7
FR-64	Review one's own saved articles, from the sidebar or from the Saved tab on one's own profile	Must	7
FR-65	Share an article or a writer profile to X, Facebook or LinkedIn, or copy its link, without needing an account	Should	7
FR-13	Receive notifications for follows, likes, comment likes, saves, comments and replies, delivered live	Must	3
FR-14	List past notifications and mark them read	Must	3

ID	Requirement	Priority	Sprint
FR-15	Upgrade to premium, and downgrade (simulated payment)	Must	5
FR-16	(A3 only) Read premium-visibility articles	Must	5
FR-17	(A3 only) Consume a daily AI token allowance	Must	2

2.2.3 Writer and eligible writer (A4, A5)

Table 2.4 – Functional requirements of the writers

ID	Requirement	Priority	Sprint
FR-18	Create a draft and edit it in a rich-text editor (TipTap, stored as JSON)	Must	2
FR-19	Have drafts saved automatically, debounced, without a save action	Must	2
FR-20	Insert images by paste, drag or file picker, uploaded directly to object storage	Must	2
FR-21	Set a cover image, excerpt and tags	Must	2
FR-22	See live word count and estimated reading time	Should	2
FR-23	Publish an article, choosing its placement: public or marketplace	Must	1
FR-24	Choose a visibility for public articles: free or premium	Must	1
FR-25	Update or soft-delete an own article	Must	1
FR-26	Switch an article from marketplace to public (one-way; the reverse is refused)	Should	5
FR-27	Converse with an AI assistant grounded in the writer's own published corpus	Must	2, 4

ID	Requirement	Priority	Sprint
FR-28	Apply an inline AI action to a text selection — reformulate, shorten, expand, simplify, improve	Must	2
FR-29	See which passages the assistant retrieved, grouped by source article	Should	4
FR-30	Insert an AI response into the document at the cursor	Must	2
FR-78	The assistant decides, per message, whether to answer in the panel or write into the document — superseding the manual insert of FR-30	Must	8
FR-79	Written text streams into the document at the cursor or over the selection, marked and protected until the writer keeps or discards it; Keep is one undo step; Discard restores what it replaced	Must	8
FR-80	Show the pipeline's real steps as they run, with counts, and a one-line recap after a write	Must	8
FR-81	The assistant is a panel docked beside the editor with Chat and Sources tabs; it collapses to a rail that carries the live step, and opens as a bottom sheet below desktop width	Should	8
FR-82	Record a prompt with the microphone (≤ 5 min) and have it transcribed into the assistant's input, editable, billed at 200 tokens a minute against the daily allowance	Should	8
FR-83	Hear each assistant reply read aloud, with a persisted mute; the control exists only when the key can speak	Could	8

ID	Requirement	Priority	Sprint
FR-84	Upload PDF, DOCX, TXT or MD reference documents (≤ 10 MB, ≤ 200 pages, 20 per writer) into a private library, with text extracted and embedded by the worker and a readable failure reason when it cannot be	Must	9
FR-85	Attach library documents to an article from the assistant and detach them; attachments are saved immediately	Must	9
FR-86	The assistant draws on attached documents for both answers and writes, cites passages inline as [Title, p. N], and shows the passages used with a link opening the file at the page	Must	9
FR-87	A document is the writer's alone: it never feeds writer memory, Portfolio Insights or search, and another account cannot read, attach or discover it	Must	9
FR-31	See the remaining AI token balance, updated after each action	Must	2
FR-32	View per-article analytics — views, unique readers, average read time, completion, engagement, retention curve	Must	3
FR-33	Rank own articles by views or engagement, server-side across the whole body of work	Should	3
FR-34	See progress toward marketplace eligibility	Must	5
FR-35	(A5 only) Publish to the marketplace with a price in credits	Must	5

ID	Requirement	Priority	Sprint
FR-36	(A5 only) Publish premium-visibility articles	Must	5
FR-37	View earnings — lifetime, per article, itemised by preview and purchase	Must	5
FR-38	Be notified when a magazine previews or purchases an article, and when earnings are credited	Must	5

Voice in this product is the *voice prompt* of FR-82, which dictates a request to the assistant; dictating a whole article is outside the scope of the project.

2.2.4 Magazine (A6, A7)

Table 2.5 – Functional requirements of the magazines

ID	Requirement	Priority	Sprint
FR-39	Register with magazine fields — name, slug, logo, website, description	Must	1
FR-40	(A6) Be stopped at a subscription wall before any marketplace surface	Must	5
FR-41	Subscribe, receiving a monthly credit allowance (simulated payment)	Must	5
FR-42	Top up credits outside the monthly grant	Should	5
FR-43	See the credit balance, the renewal date and the transaction history	Must	5
FR-44	Browse eligible writers, filtered by topic, searchable, with four sort orders	Must	3
FR-45	Read a writer evaluation report — audience, content, quality panels	Must	3
FR-46	Request AI Portfolio Insights for a writer, as an explicit action	Must	4
FR-47	Browse a writer's marketplace-listed articles with prices	Must	5

ID	Requirement	Priority	Sprint
FR-48	Unlock a preview for 10% of the price, and read the full article	Must	5
FR-49	Purchase the remainder (90%) and gain republish rights	Must	5
FR-50	See the curated library of fully purchased articles	Must	5
FR-72	Be refused when previewing, buying, switching to public, or deleting an article another magazine has already purchased exclusively	Must	5
FR-73	Publish an article it has purchased, under its own masthead — making it public and free, clearing the price, and recording the publisher	Must	5
FR-74	See the library split into what has been published and what is still waiting, and act only on the second	Must	5

Portfolio Insights generation is an explicit action rather than a side effect of opening a page, because each generation costs a model call.

2.2.5 Administrator (A8)

Table 2.6 – Functional requirements of the administrator

ID	Requirement	Priority	Sprint
FR-51	Review a queue of reports on articles, users and comments	Must	5
FR-52	Dismiss a report, delete the reported article, or ban its author	Must	5
FR-53	List and search users, filtered by role and by active/banned	Should	5
FR-54	Edit a user's plan (both directions), logged	Should	5

ID	Requirement	Priority	Sprint
FR-55	Grant marketplace eligibility manually, with an audit-log entry	Must	5

Several refusals are enforced server-side, each pinned by a test: the administrator role cannot be assigned, an administrator's role cannot be edited, nobody can edit their own role or plan, a banned account cannot be edited, and a magazine cannot be given a role.

2.2.6 Cross-cutting requirements

Table 2.7 – Cross-cutting functional requirements

ID	Requirement	Priority	Sprint
FR-56	Every article body access decision is taken server-side against the §7.4 matrix; locked content arrives as null	Must	5
FR-57	Content is classified at publish time; a flag files a pending report but never blocks publication	Should	5
FR-58	Any credit movement writes ledger rows inside one transaction, under a row lock	Must	5
FR-59	Report an article, a user or a comment	Must	5

2.3 Non-functional requirements

Each non-functional requirement is stated as a property together with the mechanism that provides it, so that it can be verified rather than asserted. Table 2.8 groups them by quality attribute.

Table 2.8 – Non-functional requirements

ID	Requirement	Mechanism
Security		
NFR-01	Passwords stored as bcrypt hashes, cost factor 12	authentication module
NFR-02	Access tokens expire after 15 minutes, refresh tokens after 7 days	signed JWTs; the refresh token is stateless, so its lifetime is the bound on its validity

ID	Requirement	Mechanism
NFR-03	Every protected route passes one composed guard stack	a single <code>@Auth()</code> decorator composing JWT, role, plan, account-type, subscription and AI-quota guards
NFR-04	Authorisation is decided server-side; the client is told the verdict	the article access matrix evaluated by the API; locked content arrives as <code>null</code>
NFR-05	Per-address rate limits on every endpoint, tighter on credential routes	60 requests/min globally, 10/min on sign-in, 5/min on registration
NFR-06	Financial operations are idempotent under retry	a unique idempotency key on every ledger transaction
NFR-07	Uploads are presigned, time-limited and never routed through the API	10-minute presigned PUT; the public bucket grants object reads only
NFR-08	Datastore ports bind to the loopback interface only	Compose port bindings on <code>127.0.0.1</code>
NFR-09	A ban takes effect at the next authentication	soft-deletion checked on sign-in, refresh and Google sign-in
NFR-10	No stored HTML is ever rendered as markup	no raw-HTML rendering anywhere in the frontend
Performance		
NFR-11	Semantic retrieval stays sub-linear as the corpus grows	HNSW index on the embedding column (<code>m = 16</code> , <code>ef_construction = 64</code>)
NFR-12	Full-text search is index-backed with weighted ranking	generated <code>tsvector</code> column with weights A/B/C and a GIN index
NFR-13	Prompt context is bounded	at most 5 retrieved passages, a memory block under 200 tokens, about 2,000 tokens in total
NFR-15	Dashboard totals and rates read pre-aggregated rows	four rollup tables refreshed by the worker
NFR-16	Analytics aggregation is incremental and idempotent	each run reads only the events since the last one
NFR-17	Expensive AI output is cached and invalidated by what changes it	portfolio insights cached 24 h and dropped when the writer publishes
NFR-18	Streaming responses are never buffered end to end	<code>nginx proxy_buffering off</code> and a 300 s read timeout on streaming routes
Reliability		
NFR-19	Every denormalised balance equals the sum of its ledger rows	three invariants asserted after every money-moving test
NFR-20	Concurrent purchases cannot double-spend	<code>SELECT ... FOR UPDATE</code> on the balance-owning row inside the transaction

ID	Requirement	Mechanism
NFR-21	Schema changes are versioned and applied before the application starts	a one-shot migration service the API and worker depend on
NFR-23	Failed background jobs retry with exponential back-off	three attempts per job
NFR-24	An AI provider outage degrades AI only	a model chain falling back from Groq to Gemini; the rest of the product is unaffected
NFR-25	Every service reports liveness and readiness	GET /health and GET /ready
Scalability		
NFR-26	The API holds no session state	identity carried by JWT claims
NFR-27	Long-running work never runs in the request path	six BullMQ queues consumed by a separate worker process
NFR-28	API and worker scale horizontally behind the proxy	stateless containers, one Compose service each
Maintainability		
NFR-29	Both code bases compile in strict mode with zero lint warnings	tsc --noEmit, eslint --max-warnings=0 in continuous integration
NFR-30	The API contract is generated from OpenAPI	client types generated from the backend's OpenAPI document
NFR-31	Business rules are tested, not asserted	792 backend tests in 51 suites; 183 end-to-end tests in 25 files
NFR-32	The build fails when schema and migrations disagree	a schema-drift check in continuous integration
NFR-33	Errors are captured in every runtime without personal data	Sentry in API, worker, Next.js server and browser
Usability		
NFR-34	Two breakpoints are designed and implemented	375 px and 1280–1440 px
NFR-35	One design system, applied consistently	39 design tokens and 87 components in Figma
NFR-36	A complete dark mode	theme switch across every screen
NFR-37	Lighthouse score above 90 and no critical accessibility violation on article pages	planned — see the general conclusion

2.4 Global use case diagram

Figure 2.1 gives the global use case diagram. It is deliberately coarse: each use case stands for a family of interactions refined in the per-sprint diagrams of Chapters 4 to 7. Every authenticated use case

includes *Sign in*; *Sign in with Google* extends it; listing on the marketplace extends publishing; unlocking a preview extends acquiring an article; grounding answers in uploaded documents extends the use of the assistant, which always includes retrieval from the writer’s own corpus.

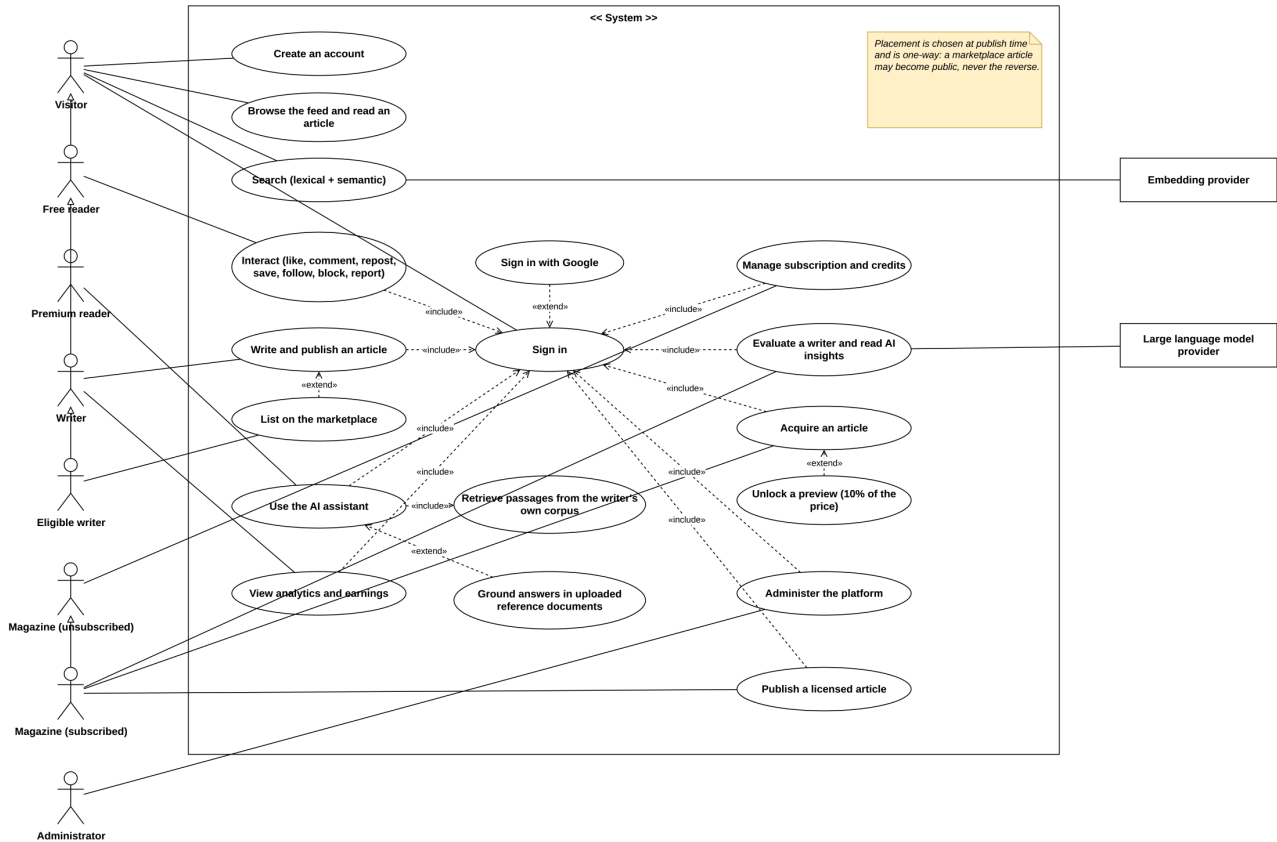


Figure 2.1 – Global use case diagram

2.5 Global class diagram

The data model counts thirty persisted tables. Drawn in a single diagram they form an unreadable strip, because the User class is related to most of the others, so the global class diagram is split by domain into two figures. Attributes are shown with domain types (Json, Vector, Timestamp); audit columns are omitted, and no methods are shown because the model is relational and its behaviour lives in the service layer.

Core domain. Figure 2.2 shows accounts, articles and their social layer: the User and its optional MagazineProfile, the Article with its placement, visibility and price, tags, threaded comments, likes, comment likes, reposts, saves, follows, blocks, notifications and reports. Article.publisherId names the magazine that published a licensed article and is empty for every other article.

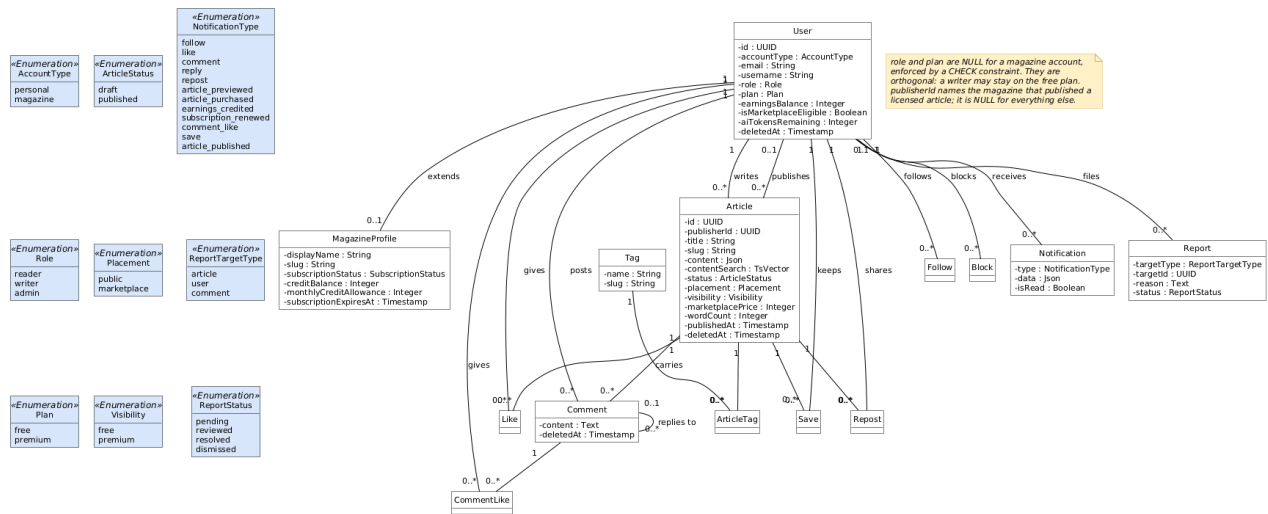


Figure 2.2 – Class diagram: core domain

AI, marketplace and analytics. Figure 2.3 shows the remaining sixteen tables: the article chunks embedded for retrieval, the writer memory, the cached portfolio insights and the AI interaction log; the private document library with its chunks and article attachments; purchases, ledger transactions, magazine subscriptions and the eligibility audit log; and the raw analytics events with the four metric rollups.

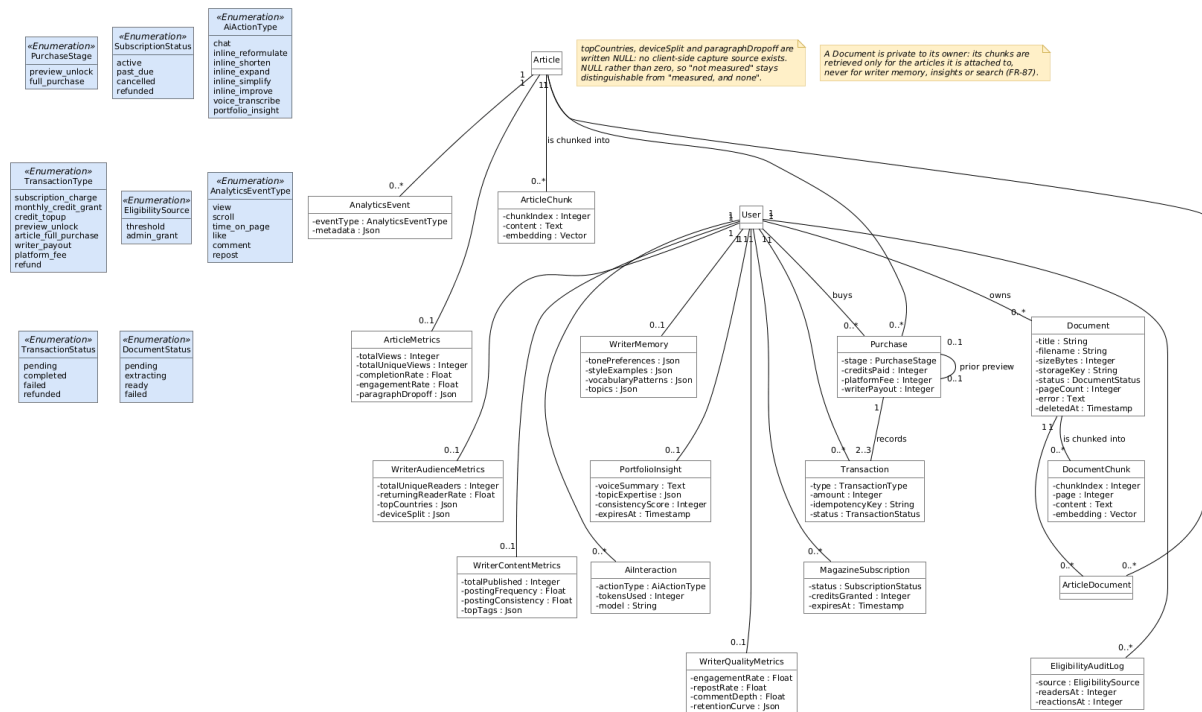


Figure 2.3 – Class diagram: AI, marketplace and analytics

2.6 Product backlog

The product backlog expresses the requirements as user stories of the form “As an *(actor)*, I want *(goal)* so that *(benefit)*”. Stories are grouped into nine epics, prioritised with MoSCoW and estimated

in story points on a Fibonacci scale relative to the simplest story (US-01). The backlog counts **72 stories for 345 points**. Table 2.9 summarises it by epic; the full backlog follows.

Table 2.9 – Product backlog by epic

Epic	Stories	Points
E1 — Accounts and authentication	5	17
E2 — Writing and publishing	8	30
E3 — AI assistance	6	38
E4 — Discovery and search	4	18
E5 — Social interactions and notifications	20	82
E6 — Analytics	4	32
E7 — Marketplace and subscriptions	12	63
E8 — Moderation and administration	6	24
E9 — Quality and operations (technical stories)	7	41
Total	72	345

Table 2.9 (continued) – The product backlog, by epic

E1 — Accounts and authentication

ID	User story	Actor	Priority	Points	Sprint
US-01	Register with an email and a password so that I can have an identity on the platform	A1	Must	3	1
US-02	Sign in with Google so that I do not manage another password	A1	Should	3	1
US-03	Register as a magazine with my branding so that writers recognise my publication	A1	Must	5	1
US-04	Stay signed in across a long writing session so that I do not lose a draft	A4	Must	3	1

ID	User story	Actor	Priority	Points	Sprint
US-05	Edit my profile and avatar so that my public page represents me	A2	Should	3	1

E2 — Writing and publishing

ID	User story	Actor	Priority	Points	Sprint
US-06	Write in a rich editor so that structure and emphasis survive publication	A4	Must	8	2
US-07	Have my draft saved as I type so that I never lose work	A4	Must	3	2
US-08	Paste or drag an image into the article so that illustrating is not a detour	A4	Must	5	2
US-09	Set a cover image so that my article looks deliberate in the feed	A4	Should	3	2
US-10	Tag an article so that readers find it by topic	A4	Should	2	1
US-11	Choose at publish time whether an article is public or for sale	A4	Must	5	1
US-12	Restrict an article to premium readers so that my best work is worth a subscription	A5	Should	3	5

ID	User story	Actor	Priority	Points	Sprint
US-13	See word count and reading time so that I can judge length as I write	A4	Could	1	2

E3 — AI assistance

ID	User story	Actor	Priority	Points	Sprint
US-14	Ask an assistant that has read my published work so that suggestions sound like me	A4	Must	13	2, 4
US-15	Reformulate, shorten, expand, simplify or improve a selection so that editing is one click	A4	Must	8	2
US-16	Insert an answer into my document so that I do not copy by hand	A4	Must	2	2
US-17	See which of my passages the assistant used so that I can trust the answer	A4	Should	5	4
US-18	See my remaining allowance so that I can pace my usage	A3	Must	2	2
US-19	Have my voice profile extracted from my corpus so that the assistant stays consistent when retrieval finds nothing	A4	Should	8	4

E4 — Discovery and search

ID	User story	Actor	Priority	Points	Sprint
US-20	Search by keyword so that I find an article I half-remember	A1	Must	5	4
US-21	Find articles that match my meaning even when they share none of my words	A1	Must	8	4
US-22	Browse by tag so that I can follow a topic	A1	Should	2	4
US-23	Find writers, not only articles, so that I can follow a person	A1	Should	3	4

E5 — Social interactions and notifications

ID	User story	Actor	Priority	Points	Sprint
US-24	Like an article so that I can signal quality	A2	Must	2	3
US-25	Comment and reply in a thread so that discussion has structure	A2	Must	5	3
US-26	Follow a writer so that I keep up with them	A2	Should	3	3
US-27	Repost an article so that my followers see it	A2	Could	2	3
US-59	React to an article from the feed so that I can engage without losing my place	A2	Must	5	8

ID	User story	Actor	Priority	Points	Sprint
US-60	See what an account has reposted so that I can judge what they rate	A2	Should	3	8
US-61	Reply in the moment from the feed so that a fresh conversation is easy to join	A2	Could	3	8
US-62	Keep reading as I scroll so that a list does not end at an arbitrary point I have to click past	A1	Must	5	8
US-65	See who a writer or magazine is from where their name appears so that I can decide to follow without leaving the feed	A1	Should	3	8
US-66	Tell the assistant what to write and watch it appear in my draft, then keep or discard it, so that generation happens where I write rather than in a chat I copy from	A4	Must	8	8
US-67	See what the assistant is doing while it works — and minimize it without losing sight of that — so that a slow step is not a frozen screen	A4	Should	3	8

ID	User story	Actor	Priority	Points	Sprint
US-68	Say my request instead of typing it so that I can brief the assistant while my hands are busy or my thoughts are faster than my typing	A4	Should	5	8
US-69	Hear the assistant's recap read out so that I can keep my eyes on the draft — and mute it when I would rather not	A4	Could	3	8
US-70	Upload the documents I am working from so that the assistant can use their facts without my retyping them	A4	Must	8	9
US-71	Choose which documents apply to an article so that a piece on tides is not fed my notes on fermentation	A4	Must	3	9
US-72	See which document and page a claim came from so that I can check it before it goes out under my name	A4	Should	5	9
US-57	Save an article to come back to it later, without anyone else seeing what I keep	A2	Must	5	7

ID	User story	Actor	Priority	Points	Sprint
US-58	Pass an article on to people outside the platform so that they can read it without signing up	A1	Should	3	7
US-28	Be told the moment someone reacts to my work, without reloading	A4	Must	5	3
US-29	Review notifications I missed so that nothing is lost between sessions	A2	Must	3	3

E6 — Analytics

ID	User story	Actor	Priority	Points	Sprint
US-30	See how many people read an article and how far they got so that I can write better	A4	Must	8	3
US-31	Rank my articles by performance so that I learn what works	A4	Should	3	3
US-32	Evaluate a writer on audience, content and quality so that I can commission with evidence	A7	Must	13	3
US-33	Read an AI assessment of a writer's voice and range so that I can shortlist quickly	A7	Must	8	4

E7 — Marketplace and subscriptions

ID	User story	Actor	Priority	Points	Sprint
US-34	Upgrade to premium so that I can read everything and use the assistant	A2	Must	5	5
US-35	Subscribe as a magazine so that I can access the marketplace	A6	Must	8	5
US-36	Receive a monthly credit budget so that spending is predictable	A7	Must	5	5
US-37	Top up credits so that a good month is not capped	A7	Should	3	5
US-38	Pay 10% to read an article in full so that I can judge before buying	A7	Must	8	5
US-39	Pay the remainder to acquire republish rights so that I can publish it	A7	Must	8	5
US-40	Keep purchased articles in a library so that my acquisitions are in one place	A7	Must	3	5
US-63	Publish an article I have bought when I am ready, rather than the purchase publishing it for me, so that my masthead stays mine to decide	A7	Must	5	5

ID	User story	Actor	Priority	Points	Sprint
US-64	Have the articles I publish appear on my public profile under my masthead so that licensing buys me a shelf and not just a file	A7	Must	5	5
US-41	List an article for sale at my own price so that I am paid for exclusivity	A5	Must	5	5
US-42	See my earnings itemised by preview and purchase so that I trust the accounting	A4	Must	5	5
US-43	See how far I am from eligibility so that the gate feels reachable	A4	Should	3	5

E8 — Moderation and administration

ID	User story	Actor	Priority	Points	Sprint
US-44	Report content so that abuse has a channel	A2	Must	3	5
US-56	Block an account so that I stop seeing someone without having to explain myself	A2	Must	5	7
US-45	Work a queue of reports so that moderation is systematic	A8	Must	5	5

ID	User story	Actor	Priority	Points	Sprint
US-46	Remove an article or ban an author so that decisions are enforceable	A8	Must	3	5
US-47	Grant eligibility manually so that a promising writer is not held back by a counter	A8	Must	3	5
US-48	Search and filter users so that I can act on the right account	A8	Should	5	5

E9 — Quality and operations (technical stories)

ID	User story	Actor	Priority	Points	Sprint
US-49	Prove that every balance equals its ledger sum, after every money-moving operation	—	Must	8	1
US-50	Bring the whole stack up with one command, reproducibly	—	Must	8	1
US-51	Fail the build on a lint error, a type error, a failing test or schema drift	—	Must	5	1
US-52	Apply migrations automatically before the application starts	—	Must	5	6
US-53	Seed a believable corpus so that the product can be demonstrated cold	—	Must	5	6

ID	User story	Actor	Priority	Points	Sprint
US-54	Lower the eligibility thresholds under a flag so that the gate can be crossed live	—	Must	2	6
US-55	Serve the product at a public URL over TLS	—	Must	8	6

2.7 Release plan

The stories are delivered in four releases of two or three sprints each, every sprint lasting two weeks. Table 2.10 gives the plan; the foundation week and the three-sprint design track are not sprints, because they deliver no user-facing increment, and are presented with the working environment in Chapter 3.

Table 2.10 – Release plan

Release	Chapter	Sprint	Dates	Goal
R1 — Functional foundation	4	Sprint 1	09/02 – 20/02	Schema, authentication, article core, ledger invariant harness
		Sprint 2	06/04 – 17/04	Editor, AI chat and inline actions, analytics event capture
R2 — Social, analytics and AI	5	Sprint 3	20/04 – 01/05	Social interactions, notifications, aggregation, dashboards
		Sprint 4	04/05 – 15/05	Retrieval-augmented generation, portfolio insights, hybrid search, writer memory
R3 — Marketplace, testing and deployment	6	Sprint 5	18/05 – 29/05	Marketplace, subscriptions, premium, moderation, administration
		Sprint 6	01/06 – 12/06	Tests, demo mode, deployment, CI/CD

Release	Chapter	Sprint	Dates	Goal
R4 — Social depth, assistant and documents	7	Sprint 7	15/06 – 26/06	Comment likes, follower lists, block, save, share
		Sprint 8	29/06 – 10/07	Live card controls, reposts tab, byline preview, assistant writing into the document, voice
		Sprint 9	13/07 – 24/07	Document library, ingestion, attachment, citations

2.8 Schedule

Figure 2.4 places the work on the project window, from 2 February to 31 July 2026: the foundation week and Sprint 1, the design track, Sprints 2 to 9 in four releases, and the writing of this report alongside the last sprints.

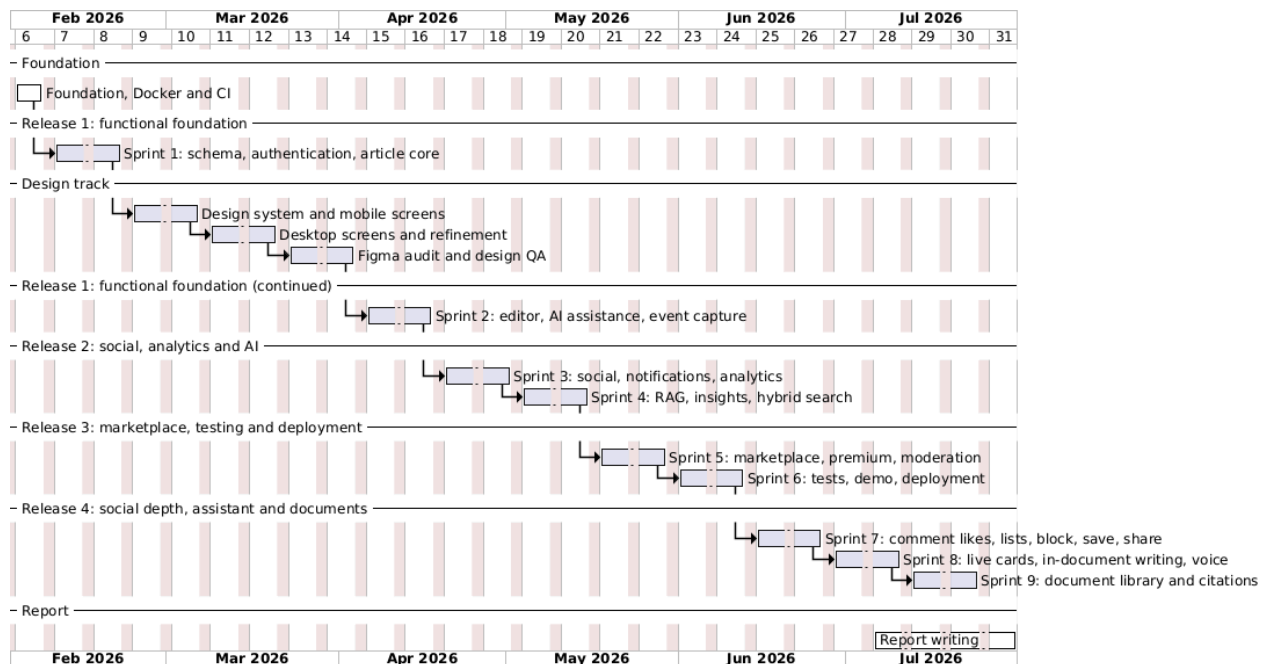


Figure 2.4 – Gantt chart of the project

Conclusion

This chapter identified the eight actors of the platform, specified its functional and non-functional requirements, presented the global use case and class diagrams, and organised the work into a backlog of 72 user stories delivered by nine sprints in four releases. The next chapter describes the environment in which the work was done and the architecture of the system.

Chapter 3 — Working environment and architecture

Introduction

Before the release chapters, this chapter describes the environment the platform was built in: the hardware and software used, the technological choices and their justification, the work that preceded the feature sprints, and the architecture of the system — its logical decomposition on the frontend and the backend, its physical deployment and its delivery pipeline.

3.1 Working environment

3.1.1 Hardware environment

[Placeholder — the development machine: processor, memory, storage, operating system. To be completed by the author.]

3.1.2 Software environment

Table 3.1 – Software tools

Tool	Use
JetBrains IDE	code editor
Git and GitHub	version control, pull requests, continuous integration (GitHub Actions) and container registry (GHCR)
Docker and Docker Compose	containers for every service, in development and in production
Figma	design system and screen design
PlantUML and draw.io	UML and architecture diagrams, kept as versioned sources
Swagger UI	exploring the REST API through the OpenAPI documentation the backend generates
Playwright	end-to-end tests and the capture of this report's screenshots

3.1.3 Foundation and design track

Two blocks of work preceded the feature sprints and are presented here rather than as sprints, because neither delivers a user-facing increment.

- **Foundation (one week).** The three repositories, their Docker images, the Compose stack, the reverse proxy and the continuous-integration pipeline were set up first, so that every later sprint started from a stack that builds, tests and runs with one command.

- **Design track (three sprints).** The user interface was designed in Figma before the editor and the social features were built: a design system of **39 tokens and 87 components**, then the full screen set at two widths — **375 px for mobile and 1280–1440 px for desktop**, 58 screens — then an audit of the screens against the specification. The release chapters implement these designs.

3.1.4 Repository organisation

The source code lives in four Git repositories. `docker.inkwell.ai` is the superproject: it holds the Compose files, the nginx configuration, the deployment workflow and the developer Makefile, and it carries the three others as **Git submodules** — `backend.inkwell.ai`, `frontend.inkwell.ai`, and `spec.inkwell.ai`, which holds the specifications, the diagrams and this report’s figures. A commit of the superproject therefore identifies one exact, deployable revision of the whole system.

3.2 Technological choices

3.2.1 Frontend

Table 3.2 – Frontend technologies

Technology	Role	Why it was chosen
Next.js 16 (App Router) with React 19	web framework	server rendering for public articles and search engines, file-system routing, a request proxy for route protection
TypeScript (strict)	language	one typed language across frontend and backend
Tailwind CSS 4 and accessible headless primitives	styling and components	implements the Figma design tokens directly; accessible dialogs, menus and tabs
TipTap 3	rich-text editor	ProseMirror-based, stores the document as JSON, extensible with the AI write range and inline actions
TanStack Query 5 and Zustand 5	server state and client state	caching, optimistic updates and invalidation for API data; small stores for UI state
Vercel AI SDK (React bindings)	assistant client	consumes the typed UI-message stream of the assistant turn
Axios	HTTP client	interceptors implement the silent token refresh

3.2.2 Backend

Table 3.3 – Backend technologies

Technology	Role	Why it was chosen
NestJS 11	API and worker framework	modular structure, dependency injection, guards and interceptors, OpenAPI generation
Drizzle ORM	data access and migrations	type-safe SQL close to PostgreSQL, explicit migrations, no hidden queries
PostgreSQL 16 + pgvector	single data store	relational data, full-text search (tsvector) and vector search (HNSW) in one database and one transaction
Redis 7 + BullMQ	queues and scheduled jobs	retries with back-off, repeatable jobs, a separate worker process
MinIO	object storage (S3 API)	images and private documents, uploaded by the browser through presigned URLs
Vercel AI SDK	AI orchestration	one interface over several providers, streaming, tool calls and typed UI streams
Passport (JWT, Google OAuth 2.0) and bcrypt	authentication	standard strategies; password hashing with cost factor 12

3.2.3 Artificial-intelligence services

Table 3.4 – AI models and their use

Use	Provider and model
Chat, writing, inline actions, portfolio insights, writer memory	Groq — gpt-oss-120b, falling back to gpt-oss-20b, then Google gemini-3.5-flash
Embeddings (articles, documents, queries)	Google gemini-embedding-001, 1,536 dimensions
Speech-to-text (voice prompt)	Groq — whisper-large-v3-turbo
Text-to-speech (replies read aloud)	Google gemini-2.5-flash-preview-tts
Content moderation at publish time	Groq classifier model

The chain of chat models is what makes a provider outage degrade the assistant only: the first chunk of a response is pulled before anything is sent to the browser, so a refusal from one model can still be answered by the next.

3.2.4 Infrastructure and quality

Docker Compose runs eight services — nginx, web, api, worker, a one-shot migration service, PostgreSQL, Redis and MinIO. **nginx** terminates TLS and routes traffic. **GitHub Actions** runs the quality gates and builds the images; **GHCR** stores them. **Jest** tests the backend against a real PostgreSQL database, **Playwright** drives a real browser end to end, and **Sentry** captures errors in the four runtimes.

3.3 Logical architecture

3.3.1 Frontend

Figure 3.1 shows the logical architecture of the frontend. The **routes** (28 pages) compose **feature modules** (18, among them the editor, the assistant, documents, saves, reposts, the marketplace and the evaluation report). A **state and data** layer holds the query cache, the client stores, the HTTP interceptors that refresh an expired access token and retry the request, and the analytics tracker. A **request proxy** protects private routes before a page renders. Two channels bypass the REST API: an **SSE client** for live notifications, and **direct uploads** from the browser to object storage through presigned URLs.

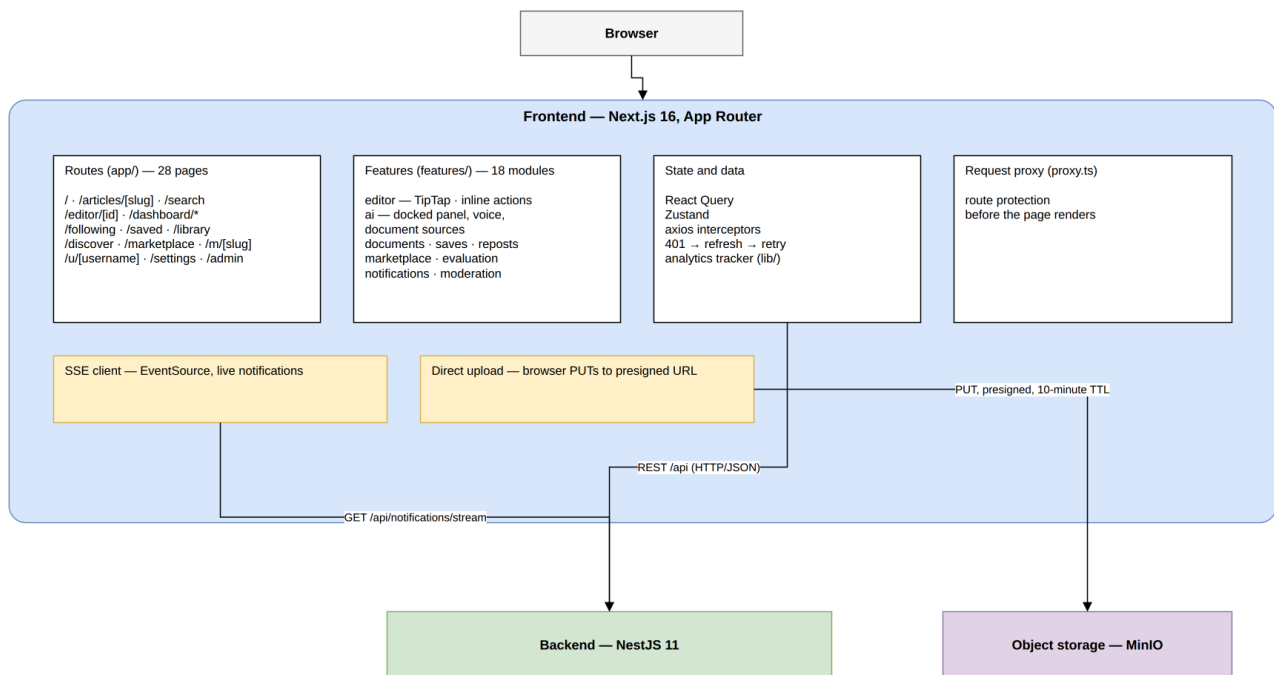


Figure 3.1 – Logical frontend architecture

3.3.2 Backend

Figure 3.2 shows the backend. The **API** is a modular NestJS application: a global rate-limiting guard runs on every route, then the guards composed once by the `@Auth()` decorator — authentication, role, plan, account type, subscription and AI quota. The feature modules cover authentication and users, articles, social interactions, AI, analytics, the marketplace, search and discovery, moderation and uploads, and documents. The **worker** runs from the same image with a second entry point and consumes six queues: embeddings (chunking and embedding articles, extracting writer memory), analytics (article and writer rollups), marketplace (eligibility and subscription renewal), ai-tokens (the nightly allowance reset), documents (extracting, chunking and embedding uploaded documents) and ai-models (a daily check that every model still answers).

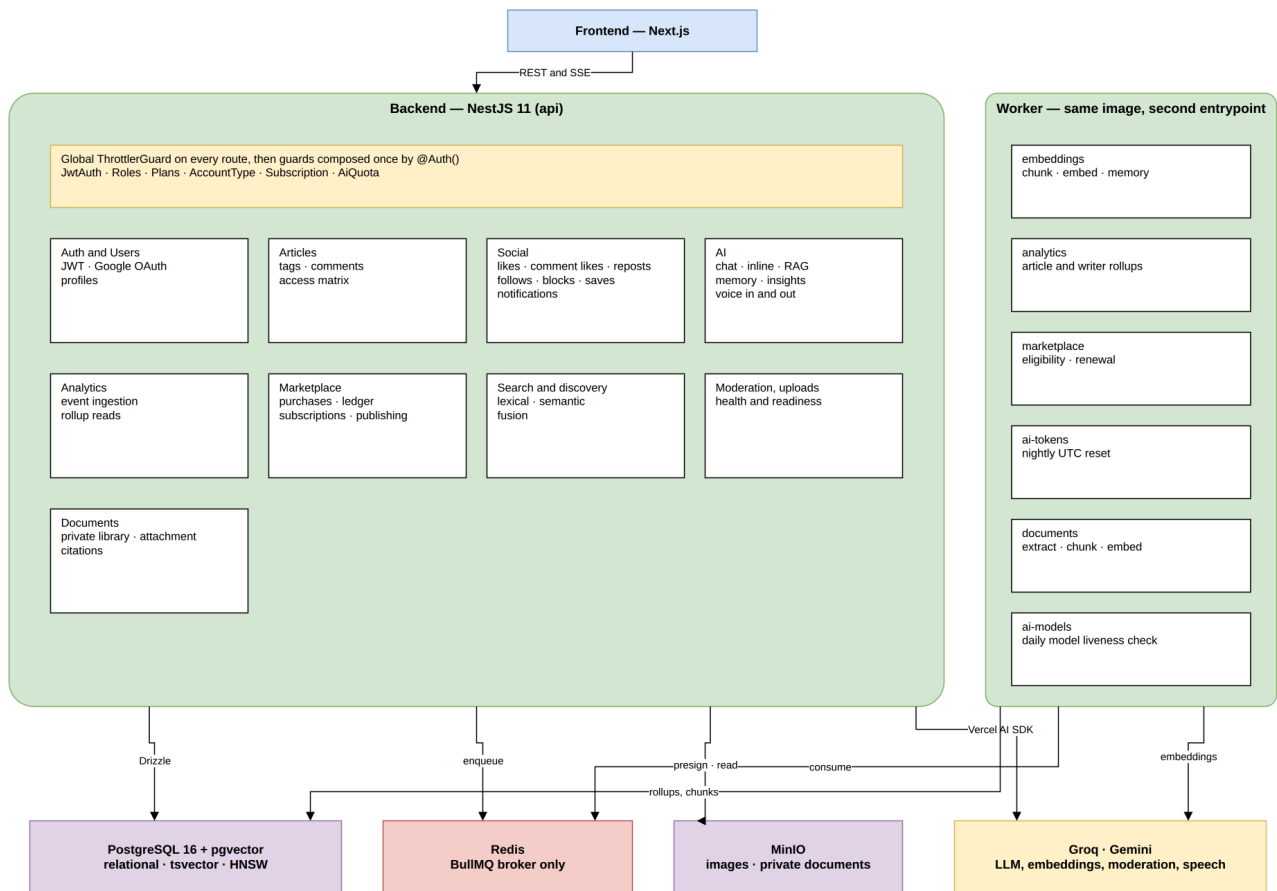


Figure 3.2 – Logical backend architecture

3.4 Physical architecture

Figure 3.3 shows the deployment. All eight services run under Docker Compose on one host. **nginx** is **the only service published**, on ports 80 and 443: it terminates TLS, proxies `/` to the Next.js server and `/api` to the NestJS API, keeps server-sent events unbuffered, and proxies the public image bucket at `/storage`. Private documents are never served through that path; they are read through the API, which checks ownership. The datastores are reachable only from inside the Compose network.

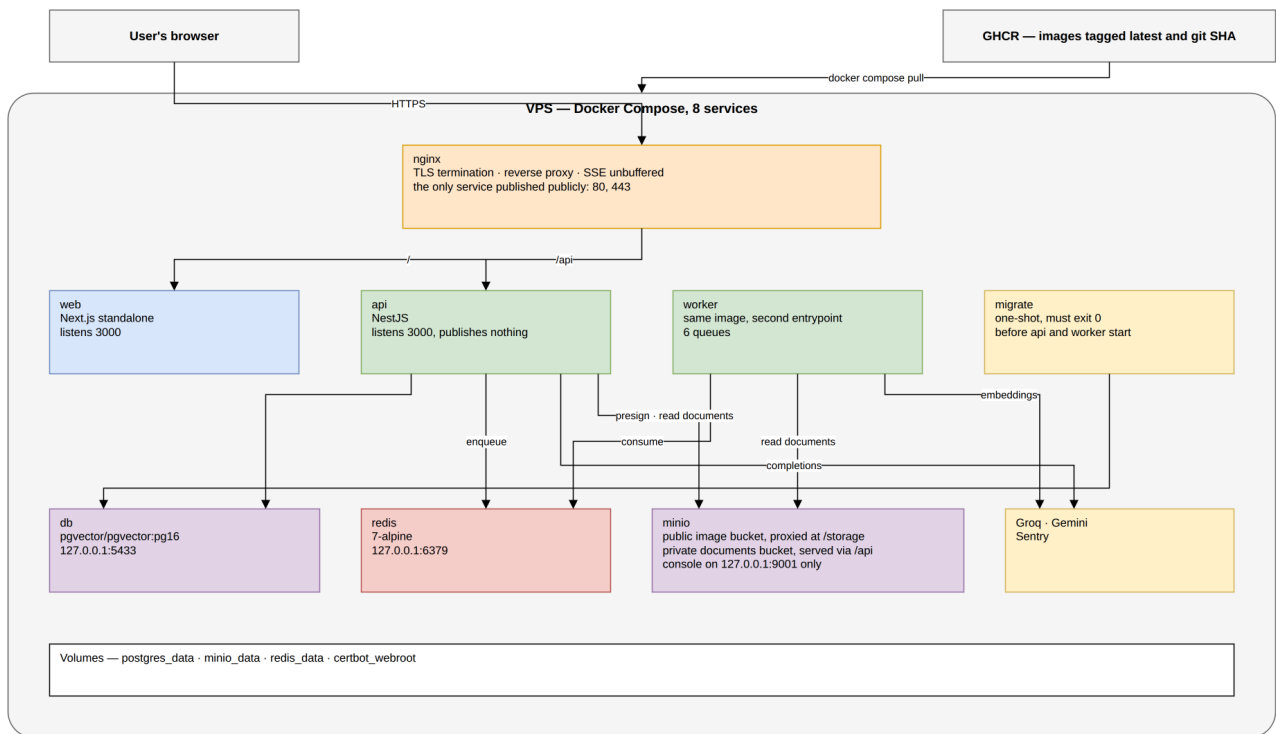


Figure 3.3 – Physical architecture

3.5 Continuous integration and delivery

Figure 3.4 shows the pipeline. A push to main on either application repository runs the quality gates — install, lint with zero warnings, type-check, the 792 backend tests, a schema-drift check — then builds a multi-stage image and pushes it to GHCR, tagged both latest and with the commit hash. A repository dispatch then triggers the deployment workflow of the superproject, which pulls the images on the server, runs the migrations (which must succeed before the API and worker start), restarts the services, verifies the health and readiness endpoints and reloads nginx.

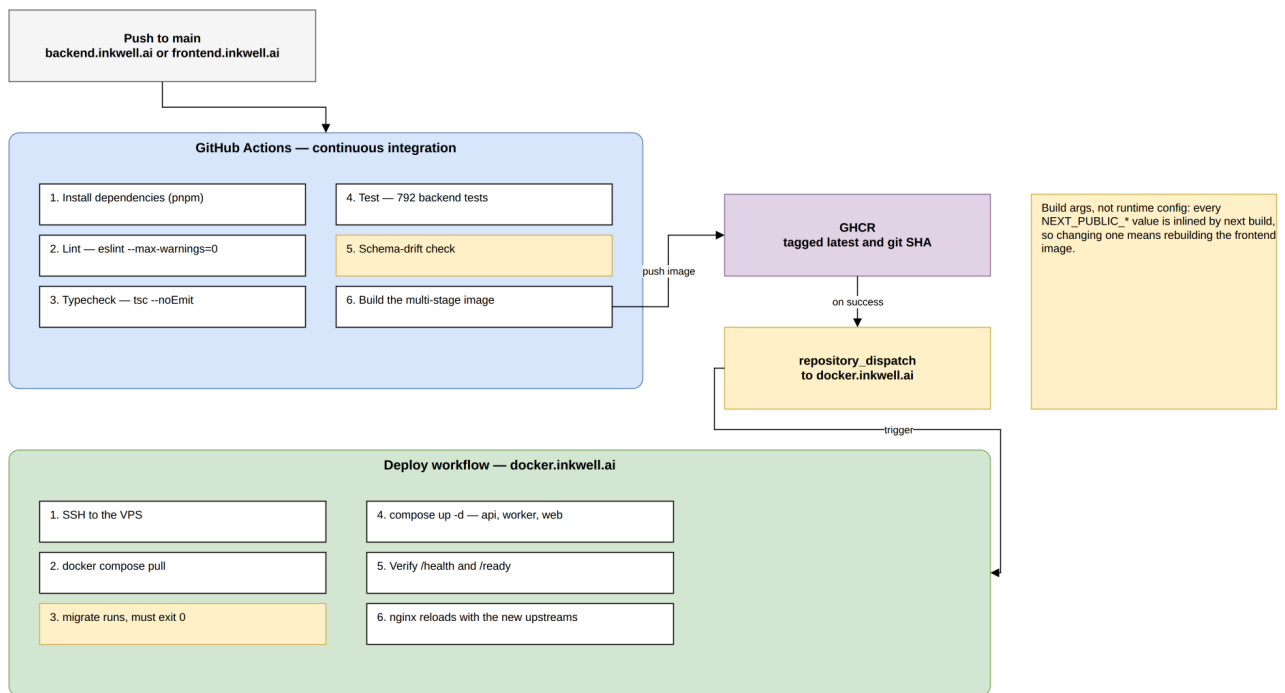


Figure 3.4 – CI/CD pipeline

Conclusion

This chapter presented the working environment, justified the technological choices and described the logical and physical architecture of Inkwell.ai and its delivery pipeline. The following chapters present the four releases, starting with the functional foundation.

Chapter 4 — Release 1: functional foundation

Introduction

The first release lays the foundation every later feature depends on. Sprint 1 delivers the data model, authentication and the core of article management; Sprint 2 delivers the rich-text editor, the first version of the AI assistant and the capture of reading events that the analytics of Release 2 will aggregate. For each sprint this chapter gives the sprint backlog, the analysis and design, and the realisation.

4.1 Sprint 1 — Schema, authentication and article core

4.1.1 Sprint goal and backlog

Goal: a visitor can register and sign in, by password or with Google; a writer can create, edit, publish and delete articles, choosing their placement and visibility; a visitor can browse the public feed and read free articles. The sprint also establishes the ledger invariant harness that every later money-moving feature is tested against.

Table 4.1 – Sprint 1 backlog

ID	User story	Priority	Points
US-01	Register with an email and a password so that I can have an identity on the platform	Must	3
US-02	Sign in with Google so that I do not manage another password	Should	3
US-03	Register as a magazine with my branding so that writers recognise my publication	Must	5
US-04	Stay signed in across a long writing session so that I do not lose a draft	Must	3
US-05	Edit my profile and avatar so that my public page represents me	Should	3
US-10	Tag an article so that readers find it by topic	Should	2
US-11	Choose at publish time whether an article is public or for sale	Must	5

ID	User story	Priority	Points
US-49	Prove that every balance equals its ledger sum, after every money-moving operation	Must	8
US-50	Bring the whole stack up with one command, reproducibly	Must	8
US-51	Fail the build on a lint error, a type error, a failing test or schema drift	Must	5
	10 stories		45

4.1.2 Analysis

Use case diagram. Figure 4.1 refines the global diagram for Sprint 1. Registering specialises into registering as a reader or as a magazine; managing one’s own articles specialises into creating a draft, editing, publishing and deleting; publishing includes choosing a placement and a visibility.

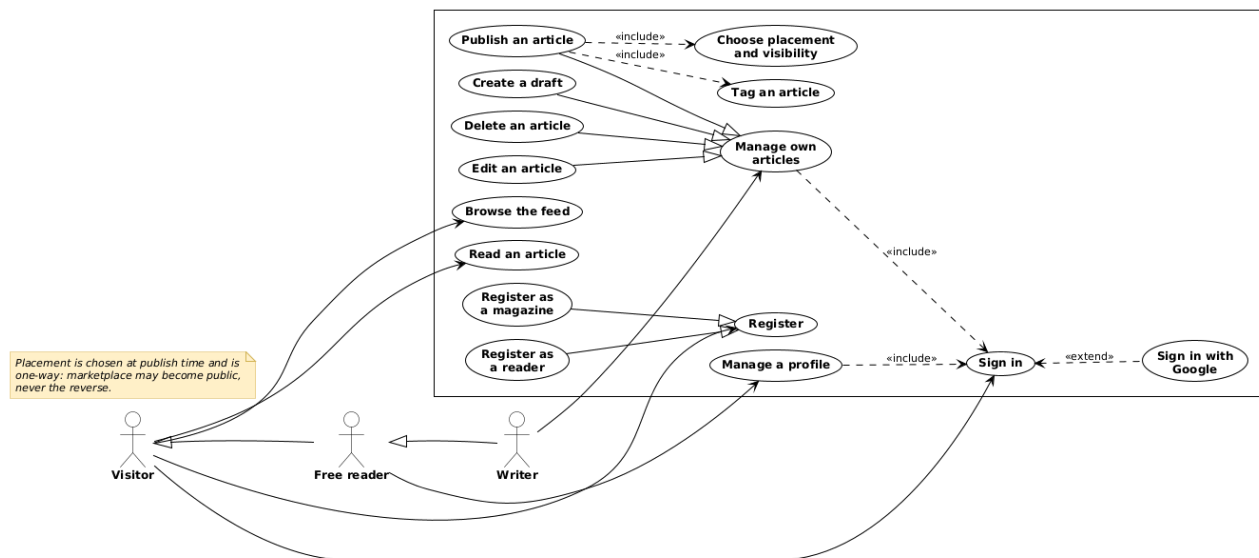


Figure 4.1 – Use case diagram, Sprint 1

Textual description. Table 4.2 details the central use case of the sprint.

Table 4.2 – Textual description of “Publish an article”

Item	Description
Use case	Publish an article
Actor	Writer (A4); eligible writer (A5) for premium or marketplace placement
Pre-condition	The writer is signed in and owns a draft

Item	Description
Post-condition	The article is published with its placement, visibility, slug and reading time
Main scenario	1. The writer opens the publish dialog. 2. The writer chooses a placement (public or marketplace) and, for a public article, a visibility (free or premium), and sets the cover, tags and excerpt. 3. The system checks that the writer may use the chosen placement and visibility. 4. The system publishes the article and computes its word count and reading time. 5. The article appears in the feed (public) or on the marketplace (marketplace).
Alternative scenario	3a. The writer is not eligible and chose premium or marketplace: the system refuses and keeps the draft. 2a. Marketplace without a price: the system refuses.

4.1.3 Design

Class diagram. Figure 4.2 shows the classes of Sprint 1: the User with its account type, authentication provider, role and plan; the MagazineProfile that extends a magazine account with its name, slug and branding; the Article with its status, placement, visibility, price and generated search vector; tags through an association table; and comments. Role and plan are empty for a magazine account, which a database constraint enforces.

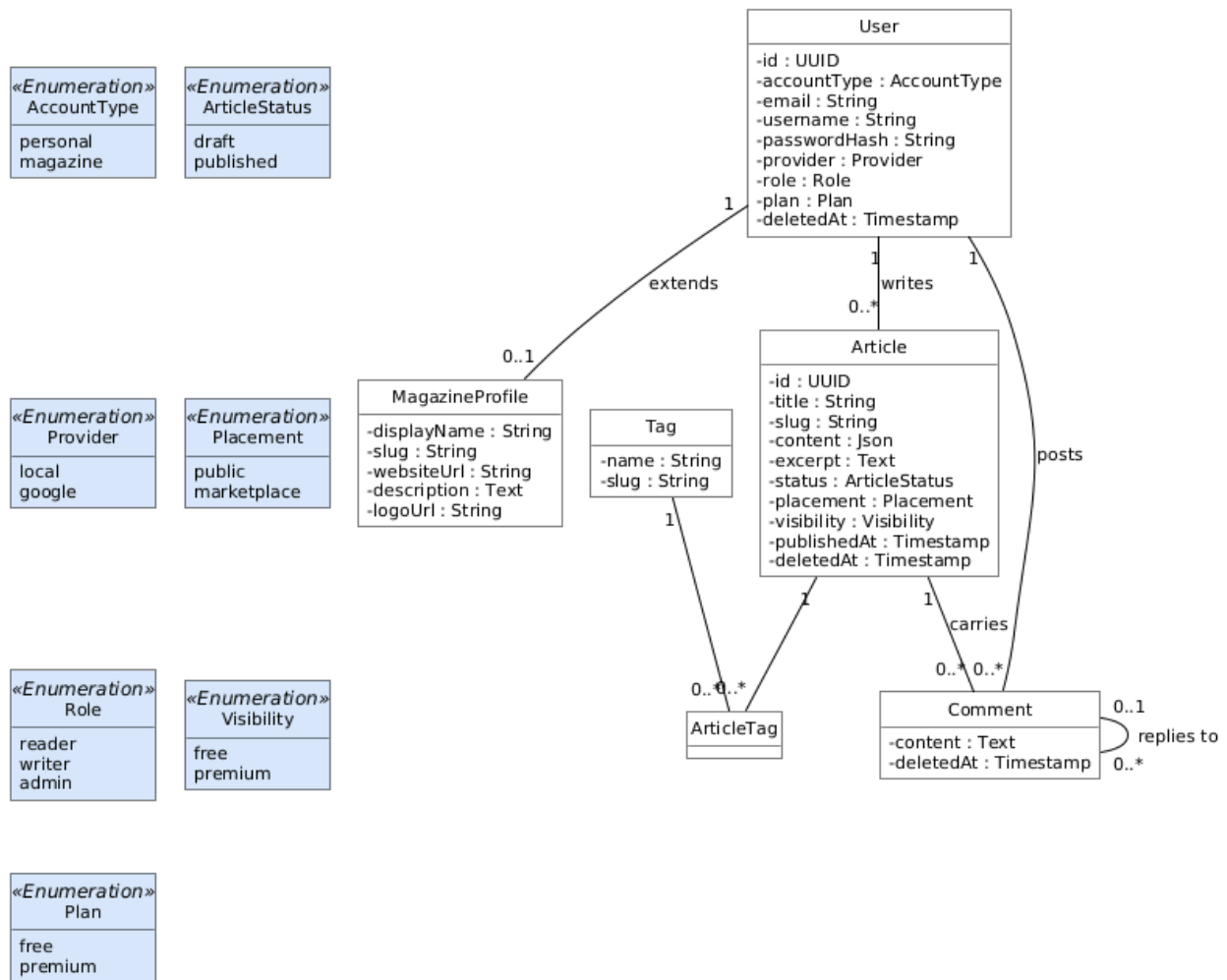


Figure 4.2 – Class diagram, Sprint 1

Sequence diagram — sign in. Figure 4.3 shows signing in and the silent refresh that keeps a session alive. Sign-in is rate-limited per address — ten attempts a minute, five for registration, against sixty for other routes — and answers 429 before the account is even read. On success the system issues a 15-minute access token and a 7-day refresh token; when a later request meets an expired access token, the client refreshes it, rewrites both the client store and the cookie the route proxy reads, and retries the original request transparently.

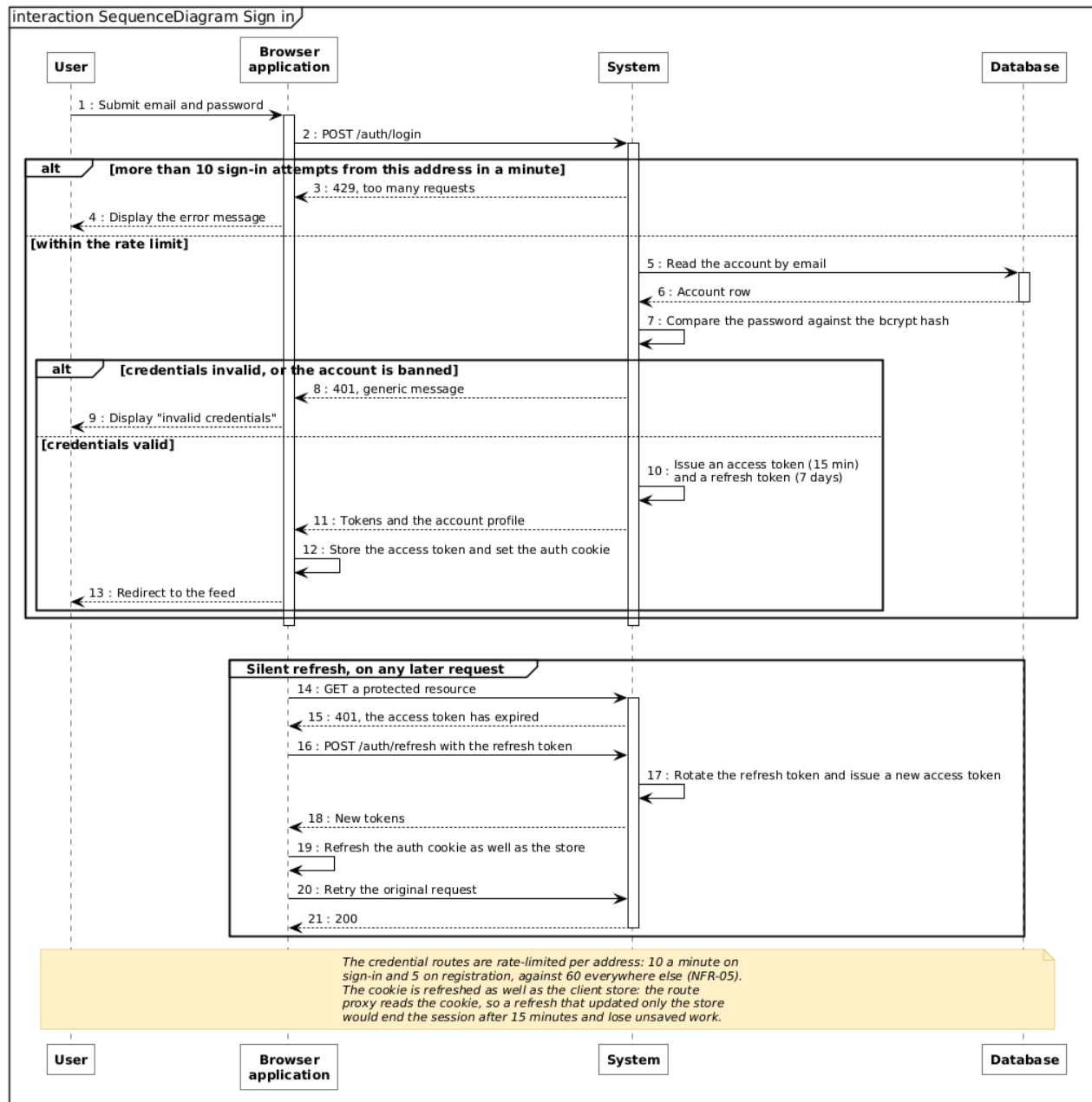


Figure 4.3 – Sequence diagram: sign in and silent refresh

The ledger invariant harness. Money-like data appears from Sprint 5 onwards, but its correctness rule is set here: every denormalised balance — a magazine’s credit balance, a writer’s earnings — must equal the sum of its ledger rows. Three invariants encode that rule and are asserted after every test that moves credits. The harness is tested by sabotage: fixtures build a coherent purchase, then corrupt one column at a time and check that the right invariant fails.

4.1.4 Realisation

 **Screenshot placeholder** — figures/screens/guest-01-home.png

Figure 4.4 – The public feed, as seen by a visitor

 **Screenshot placeholder** — figures/screens/guest-02-login.png

Figure 4.5 – Sign-in, with Google sign-in

 **Screenshot placeholder** — figures/screens/guest-03-register.png

Figure 4.6 – Registering a personal account

 **Screenshot placeholder** — figures/screens/guest-04-register-magazine.png

Figure 4.7 – Registering a magazine account

 **Screenshot placeholder** — figures/screens/guest-05-article-free.png

Figure 4.8 – A free article, readable without an account

 **Screenshot placeholder** — figures/screens/reader-04-settings.png

Figure 4.9 – Profile settings

4.2 Sprint 2 — Editor, AI assistance and event capture

4.2.1 Sprint goal and backlog

Goal: a writer drafts in a rich-text editor with automatic saving and images; a premium writer converses with an AI assistant and applies inline AI actions to a selection, within a daily token allowance; every article page records reading events.

Table 4.3 – Sprint 2 backlog

ID	User story	Priority	Points
US-06	Write in a rich editor so that structure and emphasis survive publication	Must	8
US-07	Have my draft saved as I type so that I never lose work	Must	3
US-08	Paste or drag an image into the article so that illustrating is not a detour	Must	5
US-09	Set a cover image so that my article looks deliberate in the feed	Should	3
US-13	See word count and reading time so that I can judge length as I write	Could	1
US-14	Ask an assistant that has read my published work so that suggestions sound like me	Must	13
US-15	Reformulate, shorten, expand, simplify or improve a selection so that editing is one click	Must	8
US-16	Insert an answer into my document so that I do not copy by hand	Must	2
US-18	See my remaining allowance so that I can pace my usage	Must	2
	9 stories		45

4.2.2 Analysis

Figure 4.10 gives the use cases of Sprint 2. Writing in the editor specialises into formatting, inserting an image and setting a cover image, and includes saving the draft automatically. Using the assistant specialises into chatting and applying an inline action, and includes consuming the daily allowance; inserting the answer into the document extends the chat. Reading an article includes recording reading events. Reading an article includes recording reading events.

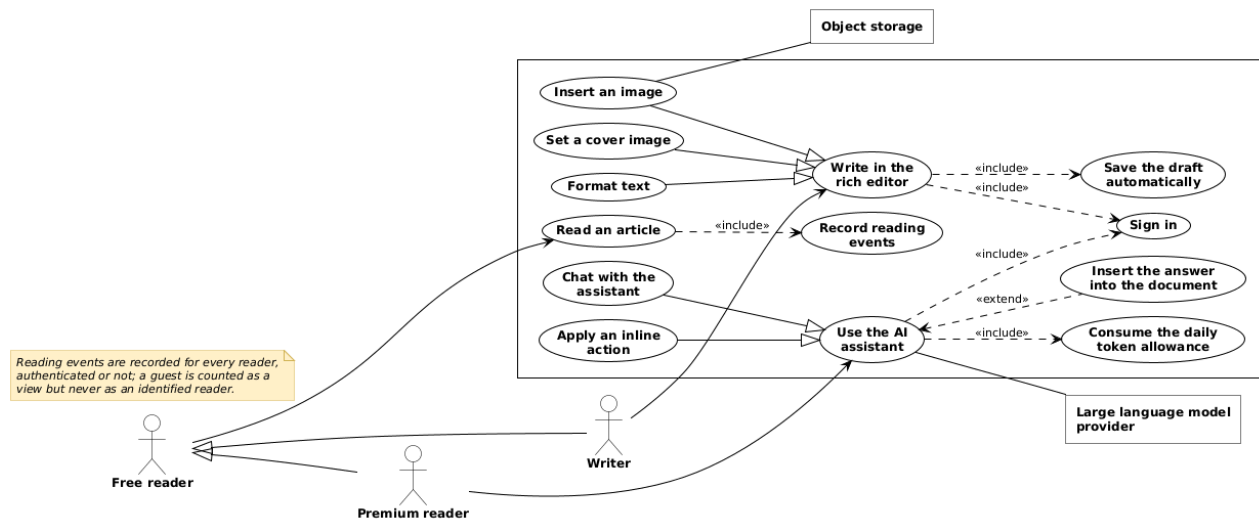


Figure 4.10 – Use case diagram, Sprint 2

4.2.3 Design

Class diagram. Figure 4.11 adds the classes of Sprint 2: each AI call is logged as an `AiInteraction` with its action type, tokens used and model; reading is recorded as `AnalyticsEvent` rows (view, scroll, time on page, like, comment, repost); and the user carries the remaining daily token allowance and the time of its last reset. The `voice_transcribe` action type is reserved in the enumeration and used from Sprint 8.

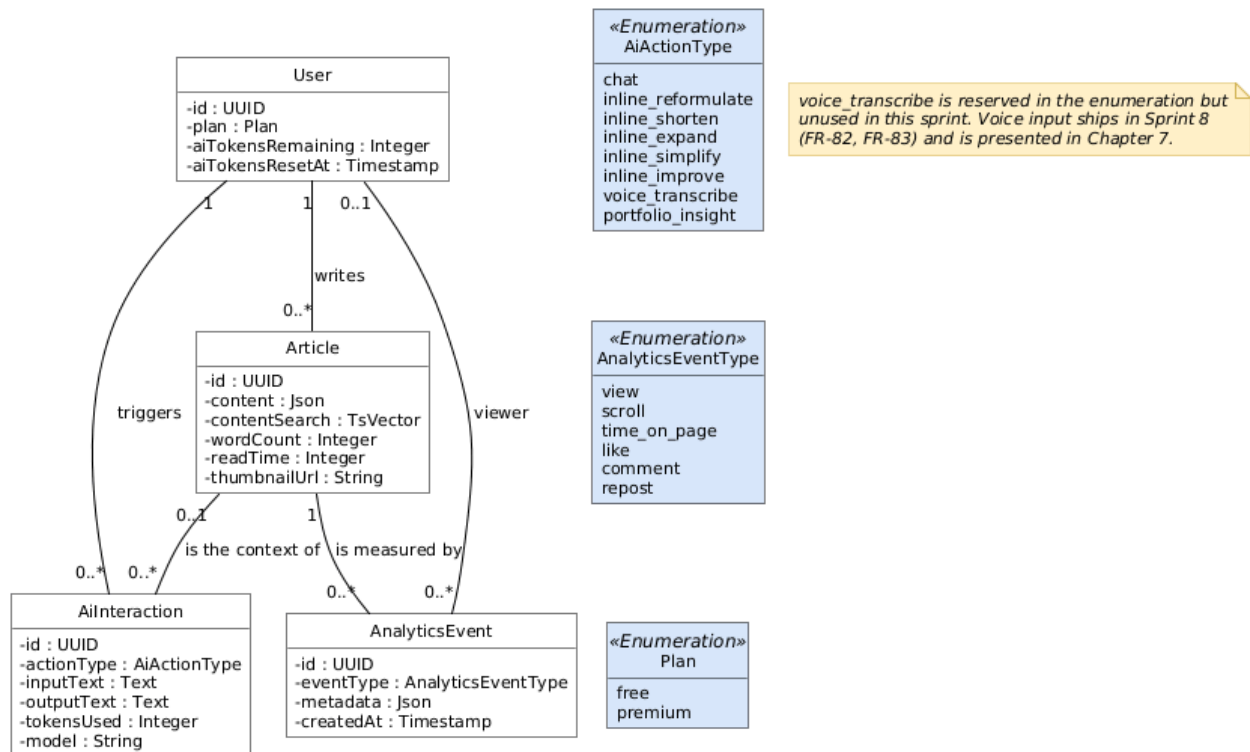


Figure 4.11 – Class diagram, Sprint 2

Sequence diagram — inline AI action. Figure 4.12 shows an inline action on a selection. The editor captures the selected range once, before anything is streamed, so that a click elsewhere cannot move it. The system checks the daily allowance and refuses with 403 and a plan-aware notice when it is exhausted; otherwise it builds the prompt from the action, the selection and the surrounding article, streams the suggestion to the editor as it is generated, and records the interaction and its cost. The allowance is checked once, before the call: a reply may cost more than what remained, and the balance then stops at zero.

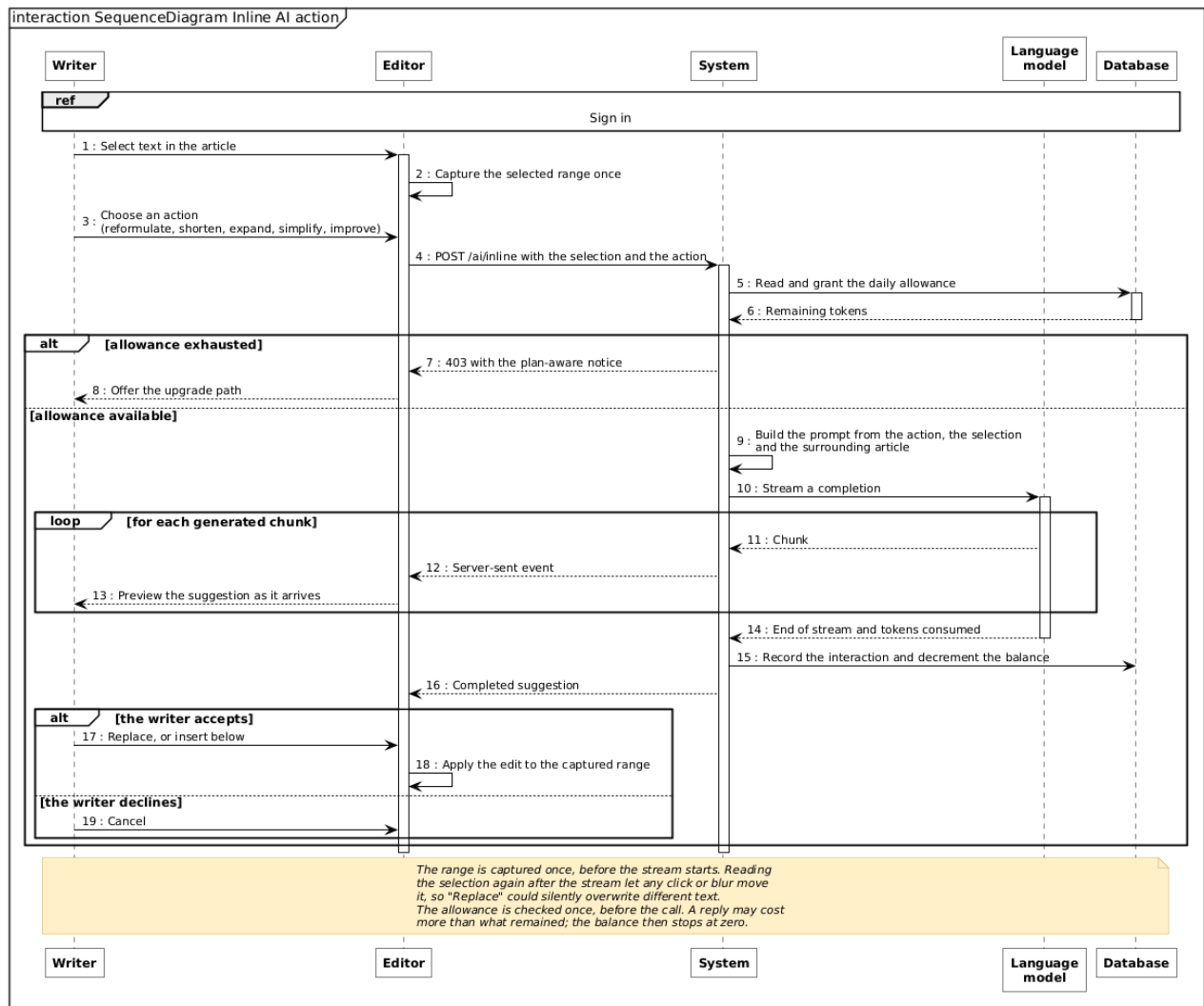


Figure 4.12 – Sequence diagram: inline AI action

Event capture. The article page observes scroll depth and engaged time and sends batched events — every ten events, every five seconds, or when the page is left — to `POST /analytics/events`. The server, not the client, attaches the reader’s identity from the access token, the country and the device. Capture starts in this sprint so that real reading data accumulates before the dashboards of Release 2 aggregate it.

4.2.4 Realisation

 Screenshot placeholder — figures/screens/writer-03-editor.png

Figure 4.13 – The rich-text editor with the assistant panel

 **Screenshot placeholder** — figures/screens/writer-04-publish-dialog.png

Figure 4.14 – The publish dialog: placement, visibility, tags and excerpt

 **Screenshot placeholder** — figures/screens/writer-05-ai-panel.png

Figure 4.15 – The AI assistant panel

 **Screenshot placeholder** — figures/screens/writer-06-ai-answer.png

Figure 4.16 – An answer from the assistant

 **Screenshot placeholder** — figures/screens/writer-07-inline-menu.png

Figure 4.17 – The inline actions on a selection

 **Screenshot placeholder** — figures/screens/writer-08-inline-suggestion.png

Figure 4.18 – An inline suggestion, ready to replace the selection

Conclusion

Release 1 delivered the foundation: accounts and authentication with rate limiting and a silent refresh, article management with placement and visibility, a rich-text editor, the first AI assistant with inline actions under a daily allowance, and the capture of reading events. Release 2 builds the social layer and the analytics on top of it.

Chapter 5 — Release 2: social interactions, analytics and AI

Introduction

The second release turns a publishing tool into a platform. Sprint 3 adds the social layer — likes, threaded comments, reposts, follows, reports — with live notifications, and aggregates the reading events captured since Release 1 into the metrics behind the writer’s dashboards and the magazine’s evaluation report. Sprint 4 adds the platform’s retrieval layer: published articles are embedded, the assistant answers from the writer’s own passages and voice profile, magazines can request AI portfolio insights, and search combines keywords with meaning.

5.1 Sprint 3 — Social interactions, notifications and analytics

5.1.1 Sprint goal and backlog

Goal: readers interact with articles and writers, and the author is told live; writers see how their articles are read; subscribed magazines browse eligible writers and read an evaluation report built on audience, content and quality metrics.

Table 5.1 – Sprint 3 backlog

ID	User story	Priority	Points
US-24	Like an article so that I can signal quality	Must	2
US-25	Comment and reply in a thread so that discussion has structure	Must	5
US-26	Follow a writer so that I keep up with them	Should	3
US-27	Repost an article so that my followers see it	Could	2
US-28	Be told the moment someone reacts to my work, without reloading	Must	5
US-29	Review notifications I missed so that nothing is lost between sessions	Must	3
US-30	See how many people read an article and how far they got so that I can write better	Must	8

ID	User story	Priority	Points
US-31	Rank my articles by performance so that I learn what works	Should	3
US-32	Evaluate a writer on audience, content and quality so that I can commission with evidence	Must	13
	9 stories		44

5.1.2 Analysis

Figure 5.1 gives the use cases of Sprint 3. Interacting with an article specialises into liking, commenting, replying, reposting, following and reporting, and includes notifying the recipient. On a marketplace listing, liking, commenting and reposting are refused while saving stays available, because a listing is a private commercial offer rather than a publication. Viewing article analytics and reading a writer evaluation both include the aggregation of raw events into metrics — a scheduled job rather than a user action, drawn because no metric exists without it.

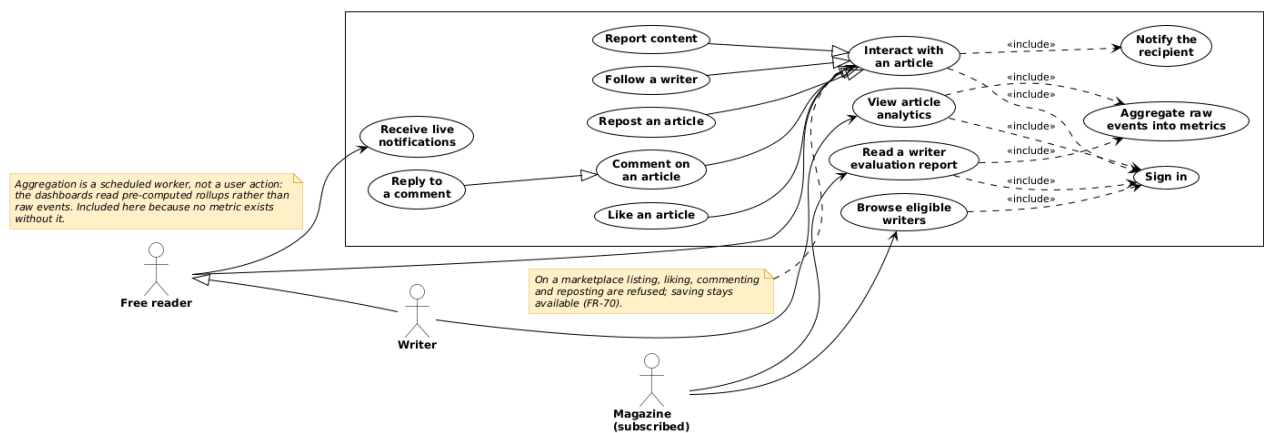


Figure 5.1 – Use case diagram, Sprint 3

5.1.3 Design

Class diagram. Figure 5.2 adds likes, comments, reposts, follows and notifications, and the four rollup tables: ArticleMetrics per article, and WriterAudienceMetrics, WriterContentMetrics and WriterQualityMetrics per writer.

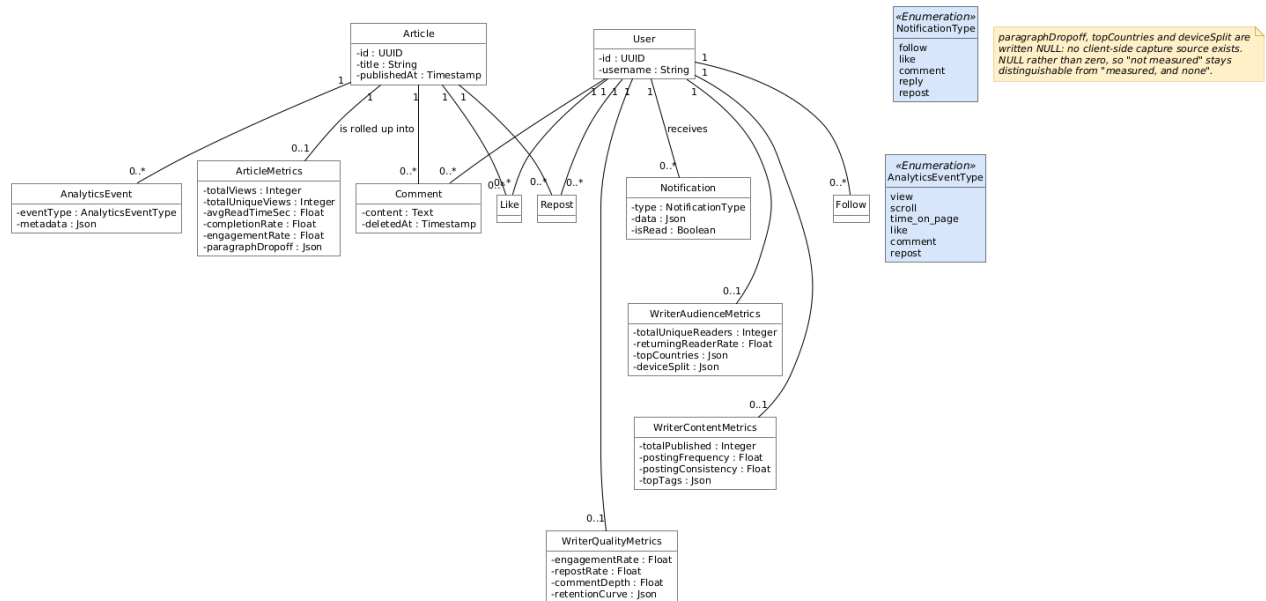


Figure 5.2 – Class diagram, Sprint 3

Sequence diagram — from reading events to metrics. Figure 5.3 follows a reading event to the dashboard. The browser batches events; the API takes the reader’s identity from the access token when one is sent — never from the request body — and attaches the country and device server-side before appending the event. Every five minutes the worker reads only the events since the last run and upserts the per-article metrics (views, unique views, reading time, completion and engagement); just after, it computes the three per-writer rollups and rechecks marketplace eligibility. Dashboards read the rollups, never the raw events, so they stay fast as the event log grows.

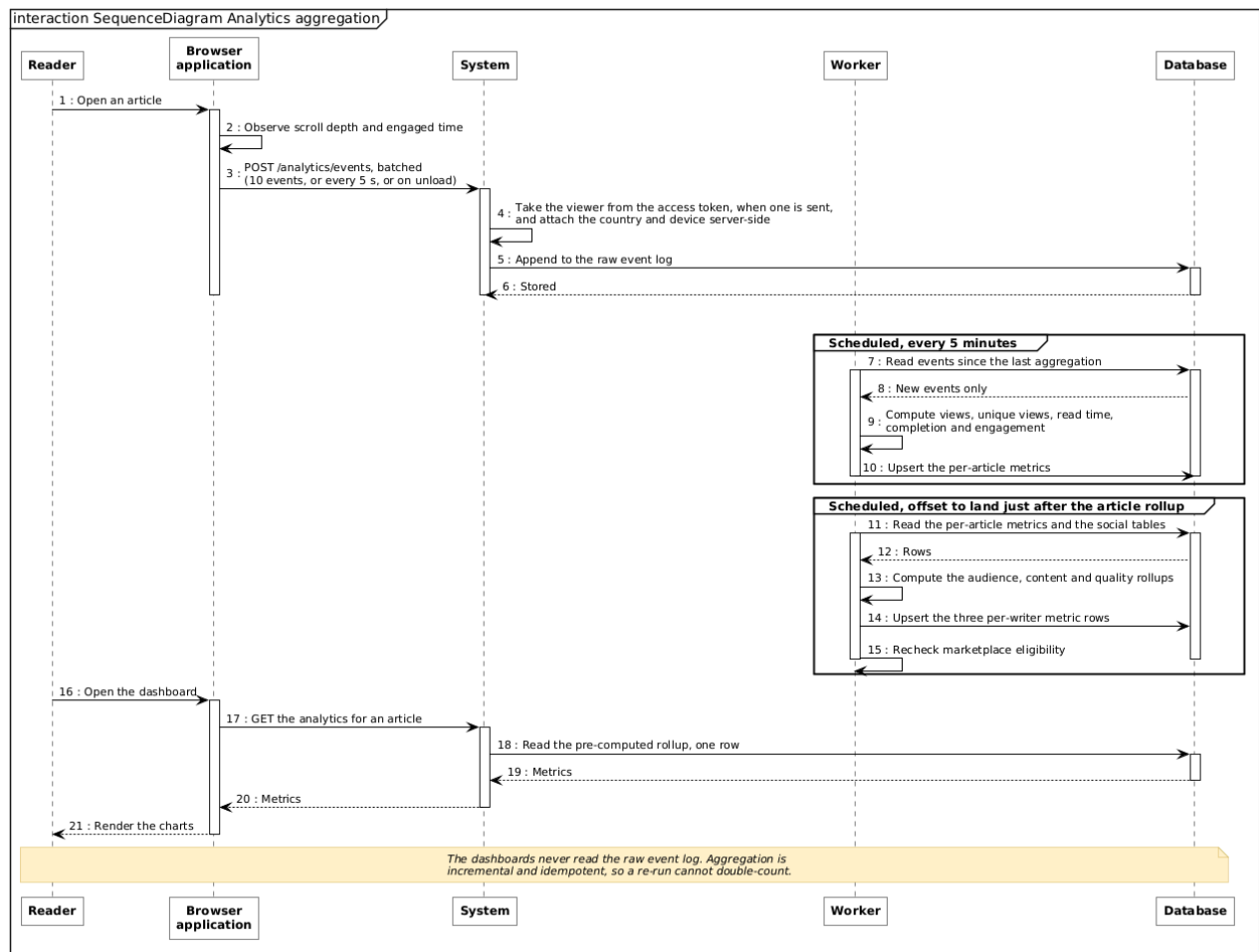


Figure 5.3 – Sequence diagram: analytics aggregation

Table 5.2 – Main metrics of the evaluation report

Panel	Metric	Definition
Audience	Unique readers	distinct identified readers across the writer's articles
Audience	Returning-reader rate	share of readers who read two or more of the writer's articles
Content	Posting frequency and consistency	articles per week; variation of the intervals between publications
Content	Topics	distribution of the writer's articles over tags
Quality	Engagement rate	share of views that read past 75% of the article
Quality	Completion rate	share of views that reach the end
Quality	Repost rate, comment depth	reposts per view; replies per comment
Quality	Retention curve	share of readers still reading at each quarter of the article

Sequence diagram — live notifications. Figure 5.4 shows how a notification reaches its recipient. The client keeps a server-sent-events stream open; when a reader likes an article, the like and the notification row are written first, and only after the commit is the event pushed to the recipient's open streams. If the connection drops, the client reconnects with the identifier of the last event received and the server replays what was missed. Delivery is best-effort; the database is the record.

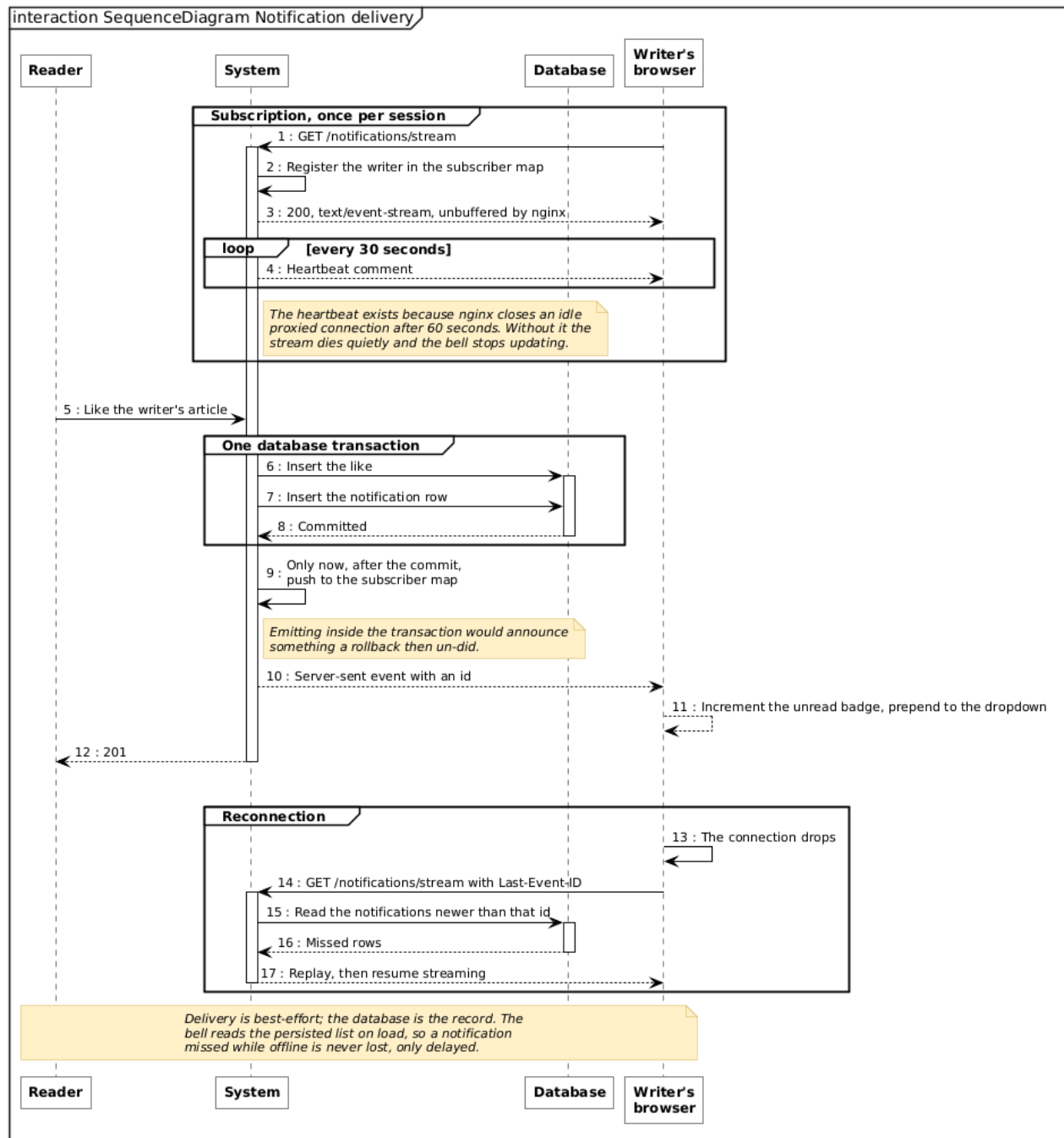


Figure 5.4 – Sequence diagram: live notification delivery

5.1.4 Realisation

 **Screenshot placeholder** — figures/screens/reader-01-feed.png

Figure 5.5 – The feed of a signed-in reader, with live card controls

 **Screenshot placeholder** — figures/screens/writer-12-notifications.png

Figure 5.6 – Notifications, filtered and grouped by day

 **Screenshot placeholder** — figures/screens/writer-01-dashboard.png

Figure 5.7 – The writer’s dashboard

 **Screenshot placeholder** — figures/screens/writer-13-my-articles.png

Figure 5.8 – “My articles”: every article, sortable by performance

 **Screenshot placeholder** — figures/screens/writer-10-dashboard-analytics.png

Figure 5.9 – The writer’s analytics: views over the last 30 days

 **Screenshot placeholder** — figures/screens/guest-07-writer-profile.png

Figure 5.10 – A writer’s public profile

 **Screenshot placeholder** — figures/screens/magazine-07-discover-writers.png

5.2 Sprint 4 — Retrieval, portfolio insights and hybrid search

5.2.1 Sprint goal and backlog

Goal: published articles become a retrievable corpus; the assistant answers from the writer's own passages and shows them; each writer gets an automatically extracted voice profile; magazines request AI portfolio insights; search finds articles by meaning as well as by keyword.

Table 5.3 – Sprint 4 backlog

ID	User story	Priority	Points
US-14	Ask an assistant that has read my published work so that suggestions sound like me	Must	13
US-17	See which of my passages the assistant used so that I can trust the answer	Should	5
US-19	Have my voice profile extracted from my corpus so that the assistant stays consistent when retrieval finds nothing	Should	8
US-20	Search by keyword so that I find an article I half-remember	Must	5
US-21	Find articles that match my meaning even when they share none of my words	Must	8
US-22	Browse by tag so that I can follow a topic	Should	2
US-23	Find writers, not only articles, so that I can follow a person	Should	3
US-33	Read an AI assessment of a writer's voice and range so that I can shortlist quickly	Must	8
	8 stories		52

5.2.2 Analysis

Figure 5.12 gives the use cases of Sprint 4. Searching specialises into searching by keyword and by meaning and includes merging the two result sets; using the assistant includes retrieving passages from the writer's own corpus and showing the sources used; publishing includes embedding the article, extracting the writer's voice profile and invalidating the insights cache; reading a writer evaluation is extended by generating portfolio insights. Insights generation is an explicit action, never a page-load side effect: each generation costs a model call, so browsing writers must not spend a magazine's budget.

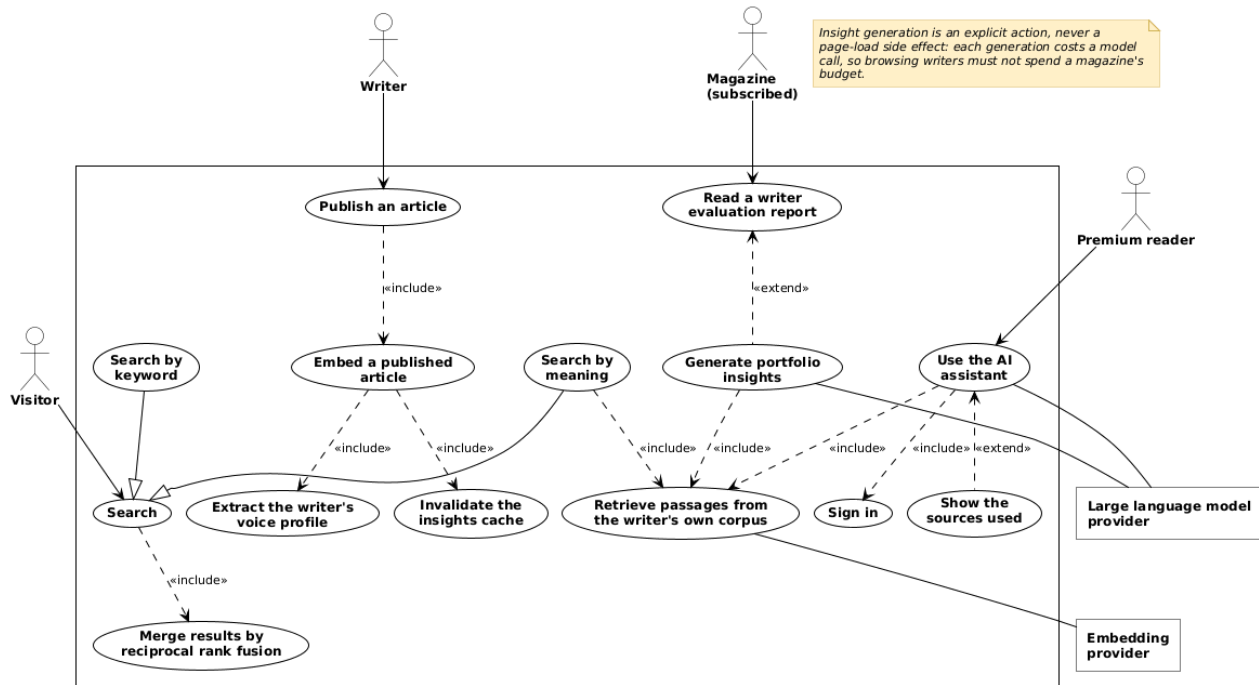


Figure 5.12 – Use case diagram, Sprint 4

5.2.3 Design

Class diagram. Figure 5.13 adds the retrieval corpus — ArticleChunk, a passage of a published article with its 1,536-dimension embedding — the WriterMemory (tone, style examples, vocabulary, topics), the cached PortfolioInsight and the AiInteraction log. Chunks exist only for published articles.

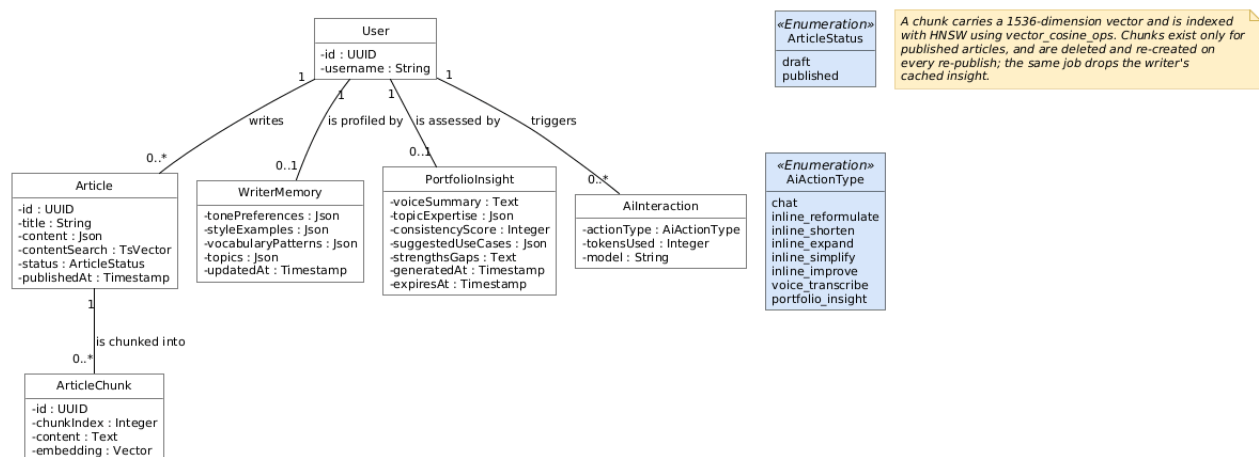


Figure 5.13 – Class diagram, Sprint 4

Sequence diagram — what publishing sets in motion. Figure 5.14 shows the background pipeline a publication starts. The API writes the article and enqueues an embedding job. The worker deletes the article’s previous chunks, splits it into passages of 120 to 1,200 characters — each prefixed with its nearest heading, so that a short paragraph embeds near its subject — embeds them, stores the vectors and drops the writer’s cached insights. It then chains the extraction of the writer’s voice profile: the ten most recent articles are sent to the model, which returns tone, style, vocabulary and topics as a schema-validated object.

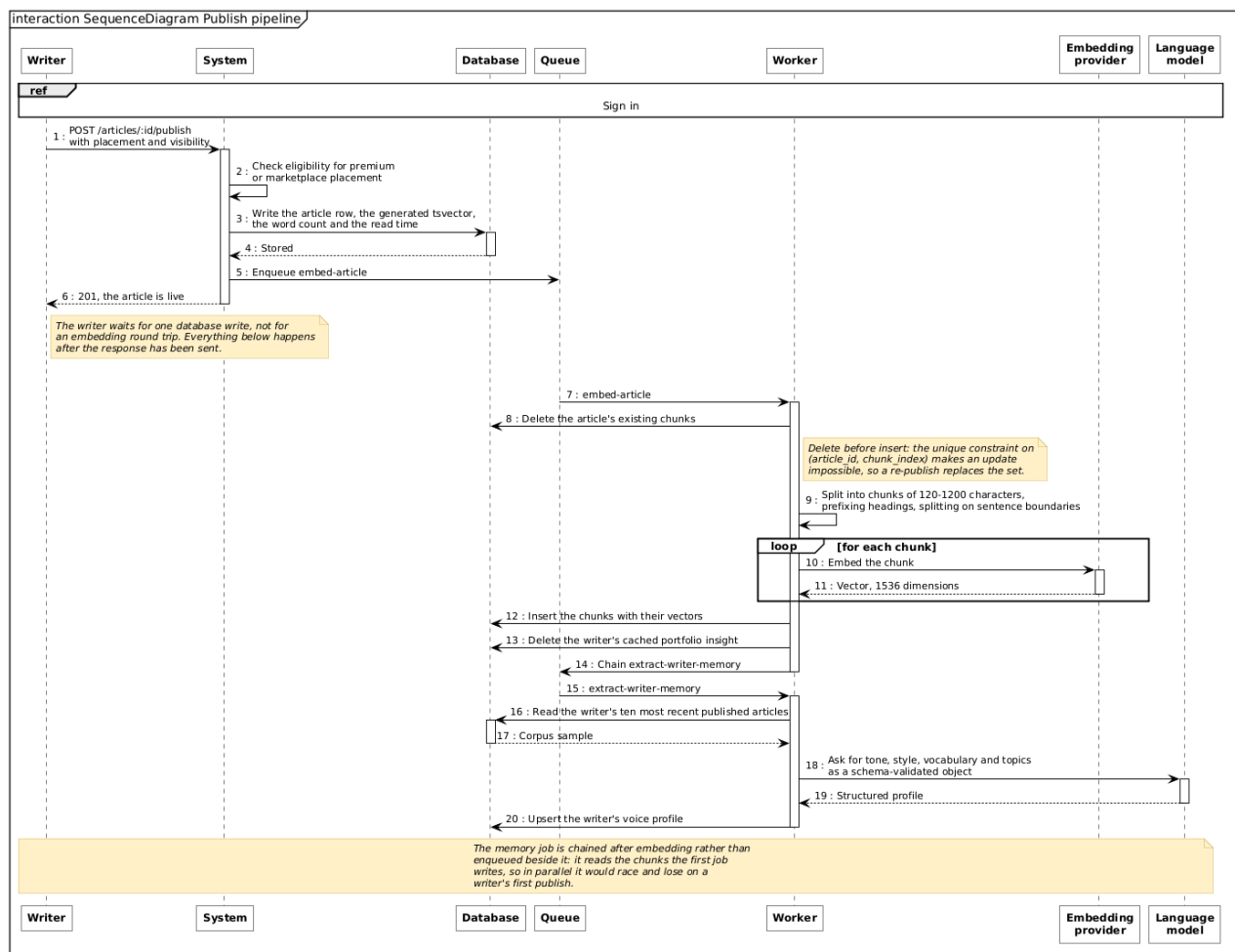


Figure 5.14 – Sequence diagram: the publishing pipeline

Sequence diagram — a chat turn grounded in the writer’s corpus. Figure 5.15 shows retrieval-augmented generation as delivered in this sprint. The question is embedded; the five passages of the writer’s own work closest to it by cosine similarity are retrieved, provided they score at least 0.60; the voice profile is read; the prompt is assembled from role, memory, passages and question, bounded at about 2,000 tokens; the answer is streamed, and the passages used are shown as sources. The similarity floor was measured rather than chosen: with this embedding model, on-topic passages scored 0.63–0.73 and off-topic ones 0.48–0.58, so a floor of 0.60 lets retrieval return nothing rather than five irrelevant passages.

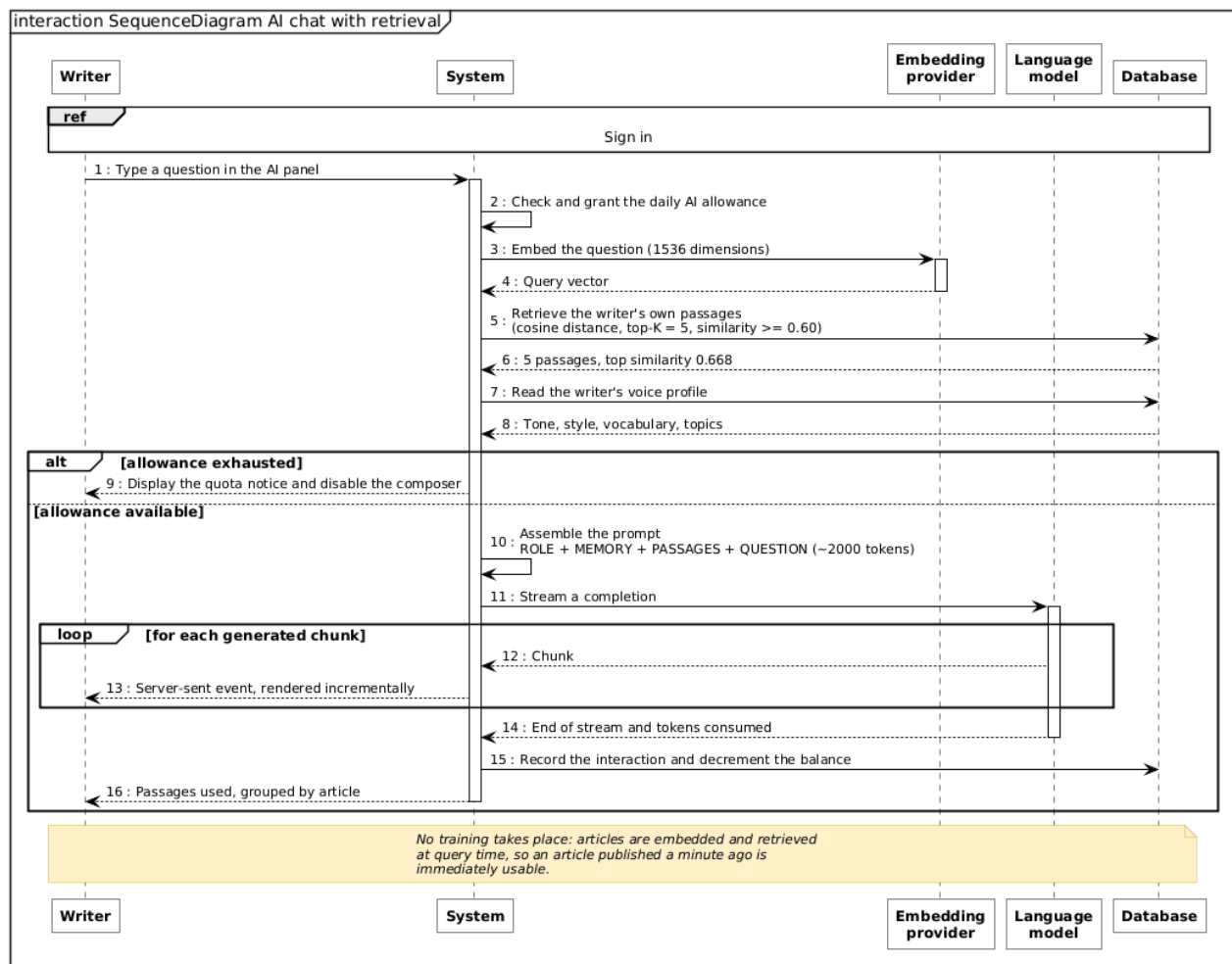


Figure 5.15 – Sequence diagram: AI chat with retrieval

Sequence diagram — hybrid search. Figure 5.16 shows search. The query is ranked lexically with ts_rank over the article’s weighted search vector, and semantically by cosine distance, keeping the best chunk per article. Because the two scores are on unrelated scales, the rankings are fused by position with reciprocal rank fusion — each article scores $\sum 1/(60 + rank)$ — so an article that never uses the searcher’s words can still be found by meaning.

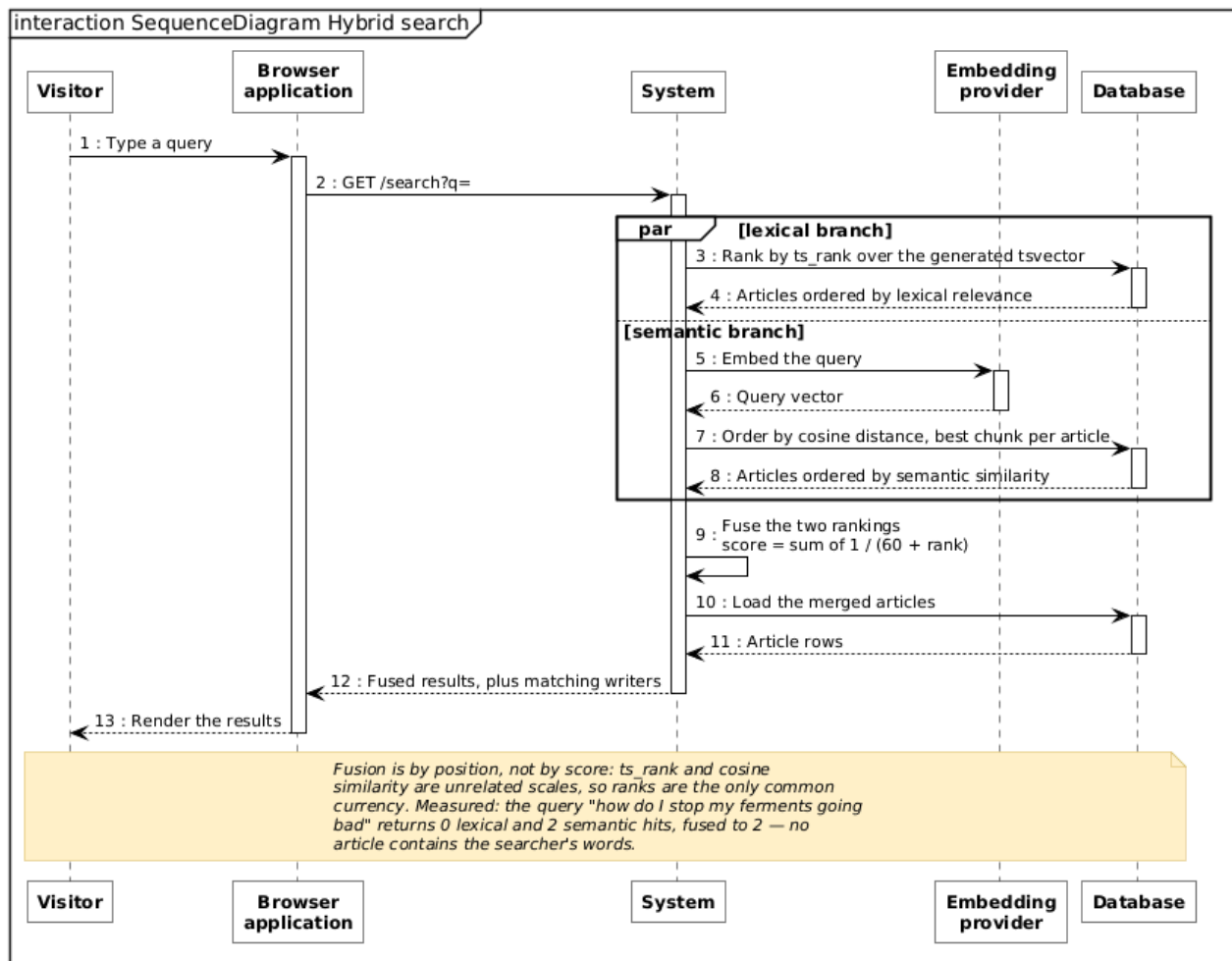


Figure 5.16 – Sequence diagram: hybrid search with reciprocal rank fusion

5.2.4 Realisation

 **Screenshot placeholder** — figures/screens/guest-09-search-results.png

Figure 5.17 – Search results

 **Screenshot placeholder** — figures/screens/magazine-08-writer-evaluation.png

Figure 5.18 – A writer evaluation report with portfolio insights

Conclusion

Release 2 made Inkwell.ai social and measurable: interactions with live notifications, reading events aggregated into per-article and per-writer metrics, an evaluation report for magazines, and a retrieval layer that grounds the assistant in the writer's own work, produces portfolio insights and powers hybrid search. Release 3 builds the marketplace those signals were designed for.

Chapter 6 — Release 3: marketplace, testing and deployment

Introduction

The third release delivers the reason the analytics exist: the marketplace. Sprint 5 adds the premium plan, magazine subscriptions and credits, the eligibility gate, the two-stage purchase with exclusive rights and the magazine's own act of publishing, and the moderation console. Sprint 6 hardens the whole: the test strategy, a demonstration corpus, and a production-ready deployment with its continuous-delivery pipeline.

6.1 Sprint 5 — Marketplace, subscriptions, premium and moderation

6.1.1 Sprint goal and backlog

Goal: readers upgrade to premium; magazines subscribe and receive monthly credits; eligible writers list articles; magazines preview, buy and publish them under their own masthead, every credit movement recorded in a ledger; writers see their earnings; administrators moderate.

Table 6.1 – Sprint 5 backlog

ID	User story	Priority	Points
US-12	Restrict an article to premium readers so that my best work is worth a subscription	Should	3
US-34	Upgrade to premium so that I can read everything and use the assistant	Must	5
US-35	Subscribe as a magazine so that I can access the marketplace	Must	8
US-36	Receive a monthly credit budget so that spending is predictable	Must	5
US-37	Top up credits so that a good month is not capped	Should	3
US-38	Pay 10% to read an article in full so that I can judge before buying	Must	8

ID	User story	Priority	Points
US-39	Pay the remainder to acquire republish rights so that I can publish it	Must	8
US-40	Keep purchased articles in a library so that my acquisitions are in one place	Must	3
US-63	Publish an article I have bought when I am ready, rather than the purchase publishing it for me, so that my masthead stays mine to decide	Must	5
US-64	Have the articles I publish appear on my public profile under my masthead so that licensing buys me a shelf and not just a file	Must	5
US-41	List an article for sale at my own price so that I am paid for exclusivity	Must	5
US-42	See my earnings itemised by preview and purchase so that I trust the accounting	Must	5
US-43	See how far I am from eligibility so that the gate feels reachable	Should	3
US-44	Report content so that abuse has a channel	Must	3
US-45	Work a queue of reports so that moderation is systematic	Must	5
US-46	Remove an article or ban an author so that decisions are enforceable	Must	3
US-47	Grant eligibility manually so that a promising writer is not held back by a counter	Must	3
US-48	Search and filter users so that I can act on the right account	Should	5

ID	User story	Priority	Points
	18 stories		85

6.1.2 Business rules

Table 6.2 – Marketplace rules

Rule	Value
Eligibility to list or publish premium	5,000 identified unique readers and 1,000 reactions (likes and comments) across the writer's public articles, lifetime; or a manual grant by an administrator. Never revoked automatically.
Magazine subscription	monthly; grants 500 credits a month; unspent credits roll over
Preview unlock	10% of the article's price, credited towards the purchase
Full purchase	the remaining 90%; exclusive — no other magazine can preview or buy the article afterwards
Platform fee	20% of each stage, rounded down; the writer receives the rest
Publication	a separate, deliberate act of the owning magazine: the article becomes public and free, its price is cleared and the magazine is recorded as publisher

6.1.3 Analysis

Use case diagram. Figure 6.1 gives the use cases of Sprint 5. An unsubscribed magazine can only subscribe; a subscribed magazine browses the marketplace, purchases an article exclusively — unlocking a preview extends the purchase — tops up credits, views its library and publishes a purchased article from it. Every credit movement includes moving credits in one transaction. An eligible writer lists an article, which includes setting a price and classifying the content at publish time; the administrator reviews the report queue, grants eligibility and manages users.

Sequence diagram — the purchase. Figure 6.3 shows the three calls of a licence. Each purchase stage opens one database transaction: it locks the article row and reads its owner, refusing with 409 if another magazine already owns it — before any arithmetic, so a refusal rolls back an empty unit of work — then locks the magazine’s credit balance, refuses with 402 when it is insufficient, debits the magazine, credits the writer, and writes the purchase and its three ledger rows. The full purchase charges only what the preview has not already paid. It leaves the article in the magazine’s library, owned and unpublished; publishing is a third call. Every request carries an idempotency key, so a retried request cannot charge twice, and the row lock stops two concurrent purchases from spending the same credits.

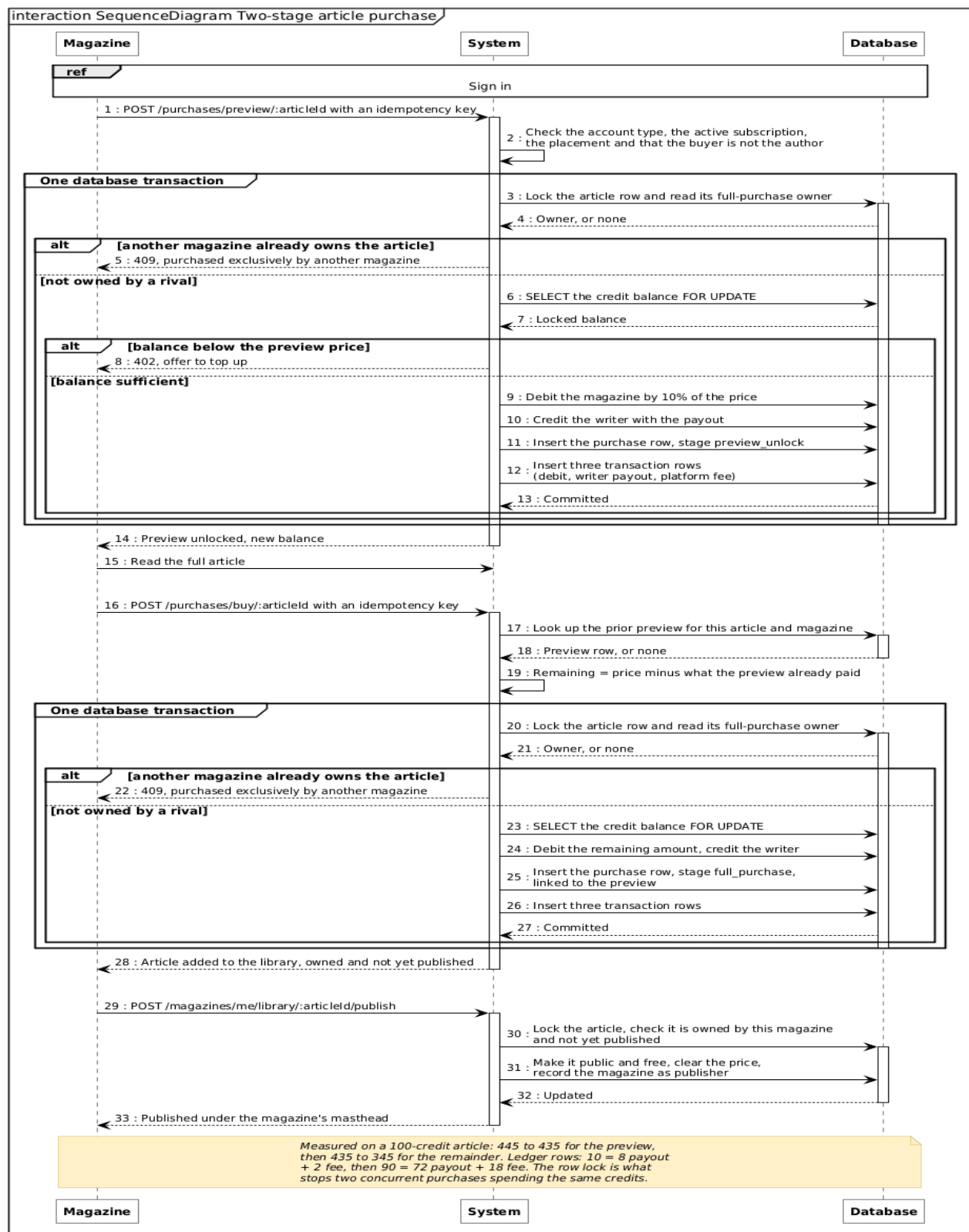


Figure 6.3 – Sequence diagram: preview, purchase and publication

Activity diagram — the eligibility gate. Figure 6.4 shows eligibility: after each aggregation the worker checks writers above both thresholds who are not yet eligible, marks them eligible and writes an audit-log entry with the metric snapshot; an administrator can grant eligibility directly, with the

same audit trail. Comment likes and reposts are deliberately not counted as reactions, so that other people's activity cannot move a writer's threshold.

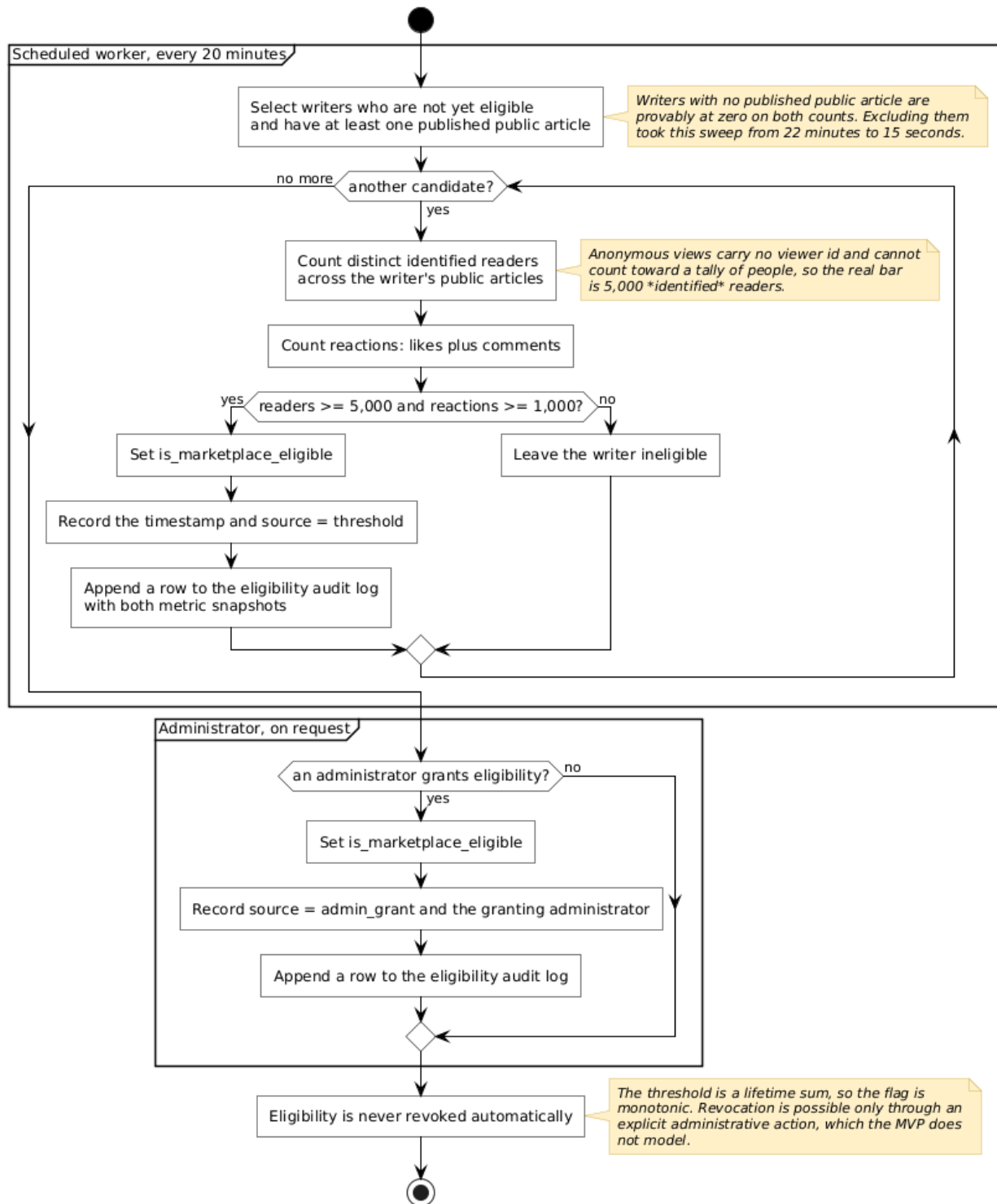


Figure 6.4 – Activity diagram: the marketplace eligibility gate

Sequence diagram — moderation. Figure 6.5 shows the two sources of reports. At publish time a classifier model reviews the article; a flag files a pending report with no reporter but never blocks

publication. Readers report articles, users or comments. The administrator works the queue: dismiss the report, remove the article or ban the author — a ban taking effect at the next authentication.

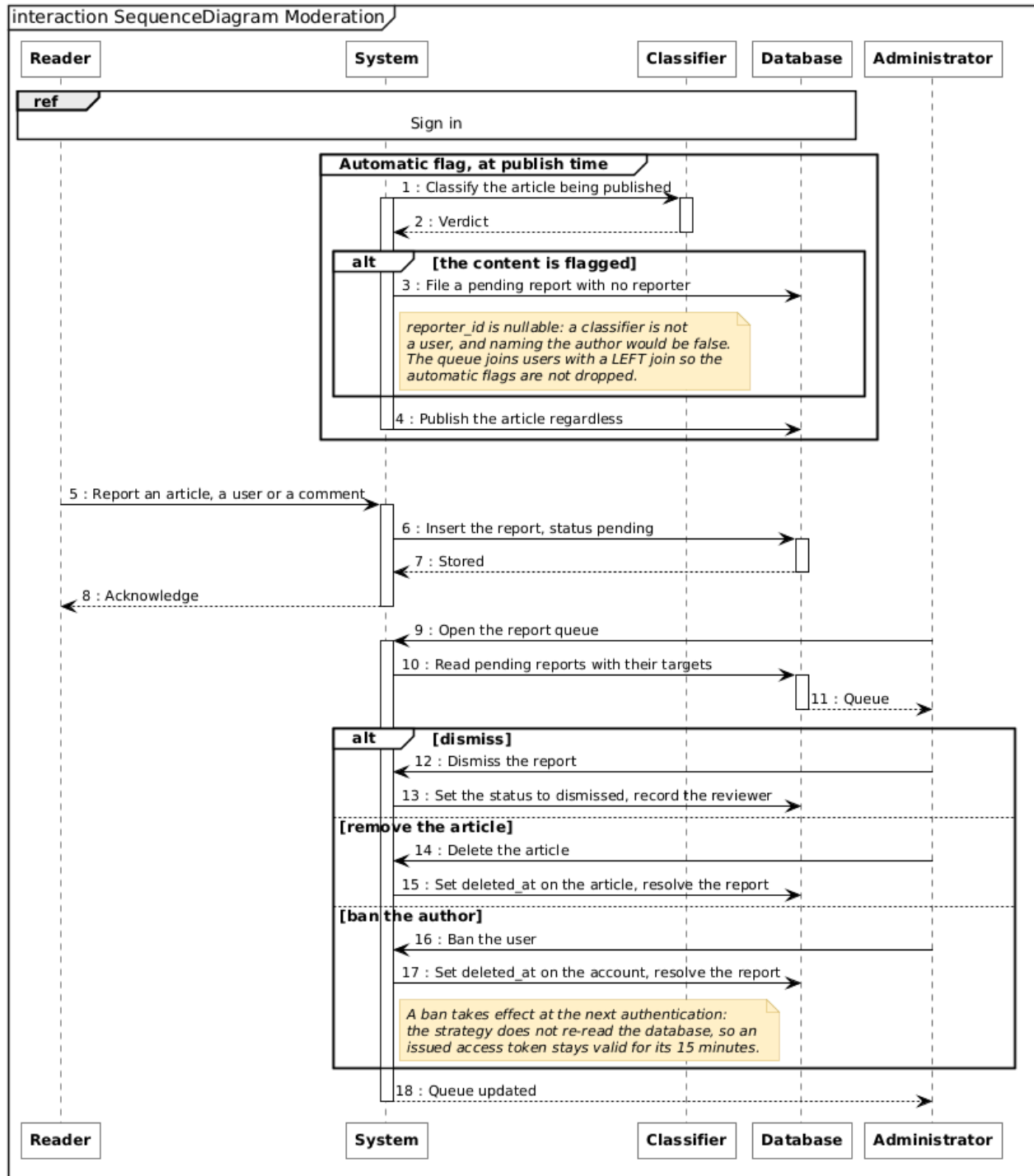


Figure 6.5 – Sequence diagram: reporting and moderation

Activity diagram — the article access decision. Figure 6.6 shows the decision taken server-side for every article body. The author and administrators always read. A marketplace article is visible only to a magazine with an active subscription, and its body only once that magazine has previewed or bought it — otherwise it receives the title, excerpt and price with the content set to null. A premium public

article requires the premium plan or a subscribed magazine. A free public article is readable by everyone, visitors included. The client is told the verdict; it never computes it.

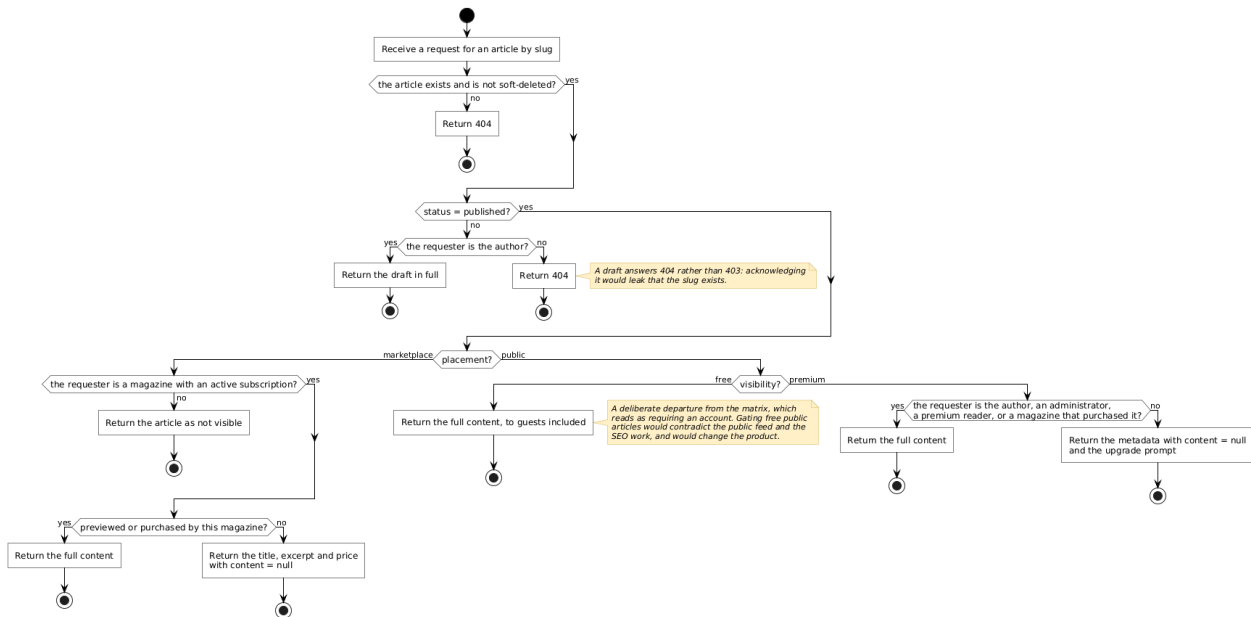


Figure 6.6 – Activity diagram: the article access decision

State diagram — the article lifecycle. Figure 6.7 shows the states an article goes through and the only transitions the API permits. A draft is published as free, premium or listed on the marketplace. A listed article can be withdrawn to public — never the reverse, because an article the public has read has no exclusivity left to sell — or deleted, as long as no magazine owns it. Its first full purchase makes it *owned*, after which the writer can neither withdraw nor delete it; the magazine's publication makes it *published by a magazine*, public and free.

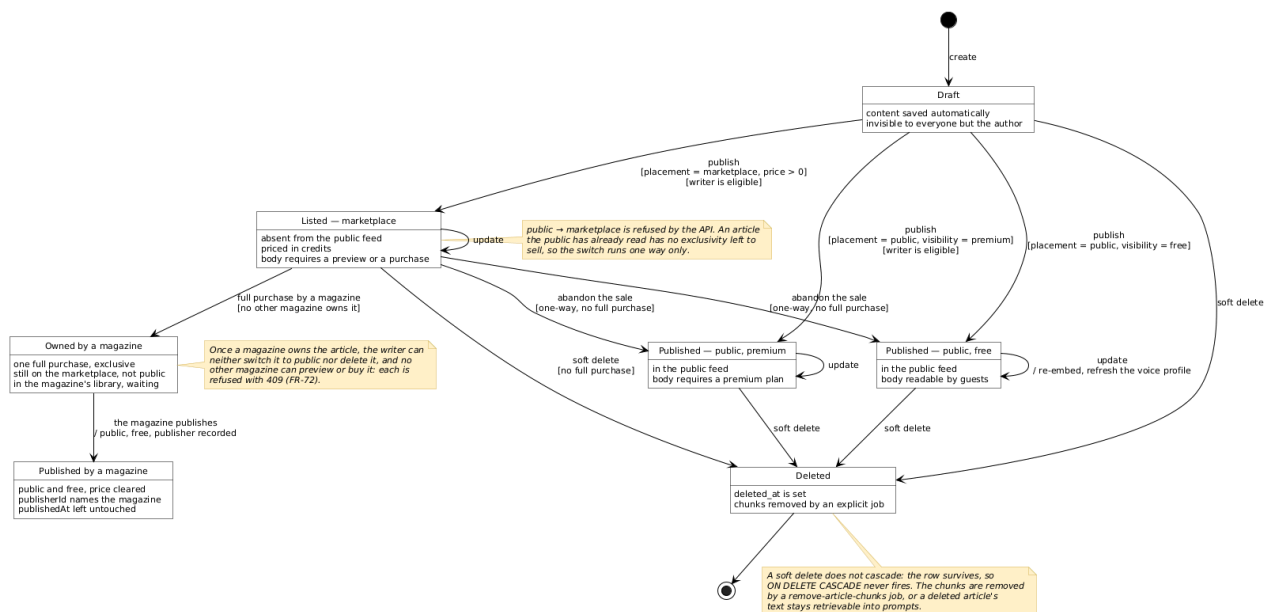


Figure 6.7 – State diagram: the article lifecycle

6.1.5 Realisation

 **Screenshot placeholder** — figures/screens/guest-06-article-premium-paywall.png

Figure 6.8 – A premium article behind the paywall

 **Screenshot placeholder** — figures/screens/reader-03-subscription-plans.png

Figure 6.9 – The plans: free and premium

 **Screenshot placeholder** — figures/screens/magazine-06-subscription-credits.png

Figure 6.10 – A magazine's subscription and credits

 **Screenshot placeholder** — figures/screens/magazine-01-marketplace.png

Figure 6.11 – The marketplace, with preview and purchase prices

 **Screenshot placeholder** — figures/screens/magazine-02-listing-detail.png

Figure 6.12 – A listing before preview

 **Screenshot placeholder** — figures/screens/magazine-03-purchase-confirm.png

Figure 6.13 – Confirming a preview unlock: price and balance after

 **Screenshot placeholder** — figures/screens/magazine-04-library.png

Figure 6.14 – The magazine’s library: unpublished and published

 **Screenshot placeholder** — figures/screens/magazine-09-publish-from-library.png

Figure 6.15 – Publishing a purchased article under the magazine’s masthead

 **Screenshot placeholder** — figures/screens/guest-10-licensed-article.png

Figure 6.16 – A licensed article naming both the writer and the magazine

 **Screenshot placeholder** — figures/screens/guest-08-magazine-profile.png

Figure 6.17 – A magazine’s public profile and its published articles

 **Screenshot placeholder** — figures/screens/writer-09-earnings.png

Figure 6.18 – A writer’s earnings, itemised by preview and purchase

 **Screenshot placeholder** — figures/screens/writer-11-sold-article.png

Figure 6.19 – A sold article, as its author sees it

 **Screenshot placeholder** — figures/screens/admin-01-eligibility-grant.png

Figure 6.20 – Administration: manual eligibility grant

 **Screenshot placeholder** — figures/screens/admin-02-user-table.png

Figure 6.21 – Administration: user management

 **Screenshot placeholder** — figures/screens/admin-03-report-queue.png

Figure 6.22 – Administration: the report queue

6.2 Sprint 6 — Tests, demonstration and deployment

6.2.1 Sprint goal and backlog

Goal: prove the business rules with tests, make the product demonstrable on a realistic corpus, and make it deployable with one command and an automated pipeline.

Table 6.3 – Sprint 6 backlog

ID	User story	Priority	Points
US-52	Apply migrations automatically before the application starts	Must	5
US-53	Seed a believable corpus so that the product can be demonstrated cold	Must	5
US-54	Lower the eligibility thresholds under a flag so that the gate can be crossed live	Must	2
US-55	Serve the product at a public URL over TLS	Must	8
	4 stories		20

6.2.2 Test strategy

Figure 6.23 shows the strategy. The base is the **unit and service tests** of the backend, which run against a real PostgreSQL database rather than mocks, so that constraints, transactions and row locks are exercised for real. Above them, **integration tests** go through the API — authentication, publishing, purchasing — and verify the ledger afterwards. At the top, **end-to-end tests** drive a real browser over a seeded corpus. Two families protect the money: the **ledger invariant harness**, whose tests are mostly negative — a fixture builds a coherent purchase, corrupts one column and checks that the right invariant fires — and a **price sweep** that checks every price from 1 to 300 credits, both stages and both splits, reconciling exactly. The harness was validated by sabotage: removing a filter from the earnings query or a stage from the debit list fails exactly the tests that should fail, an exercise that found one coverage gap which was then closed.

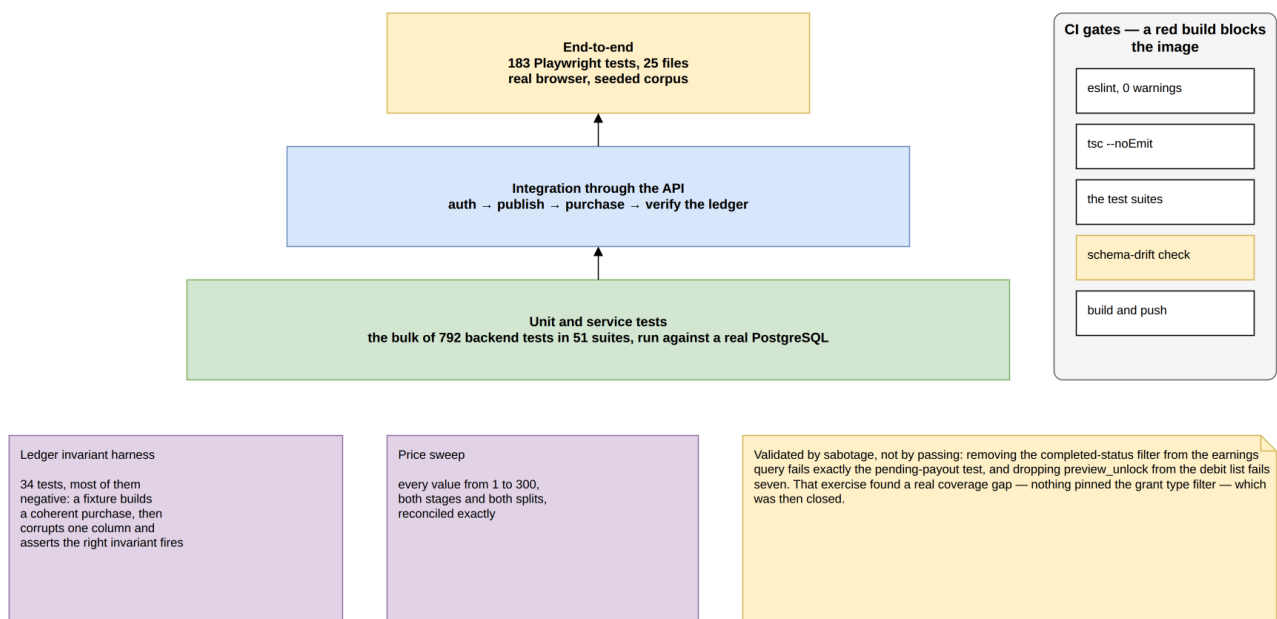


Figure 6.23 – Test strategy

Table 6.4 – Test results

Suite	Scope	Result
Backend (Jest)	51 suites — services, controllers, ledger, purchases, documents, voice, AI pipeline	792 tests, all passing
Ledger invariant harness	balance and ledger invariants, seed ledger	34 tests, included above
Price sweep	prices 1–300, both stages, both splits	included above
End-to-end (Playwright)	25 spec files over the seeded corpus	183 tests
Static gates	tsc --noEmit and eslint --max-warnings=0, both repositories	pass

6.2.3 Demonstration corpus

A seed command builds a believable dataset in seconds and can rebuild it from scratch: its full preset creates **100 writers and 989 articles**, **50 magazines** (30 with a live subscription), **150 readers**, 110 marketplace listings with real purchases and ledger rows, and more than 15,000 likes, 2,700 reposts and 8,000 comments, with generated cover art and avatars uploaded to object storage. A fixed cast of named accounts makes every demonstration path reproducible, and a flag lowers the eligibility thresholds so that the gate can be crossed live during a demonstration. Every seeded date is relative to the seed’s clock and can be bounded below, so the corpus never predates the platform.

6.2.4 Deployment

The production stack is ready to run on a single server:

- **multi-stage images** for the API and worker (one image, two entry points) and for the Next.js server in standalone mode;
- a **production Compose file** in which only nginx is published, on ports 80 and 443, with TLS certificates from Let's Encrypt renewed through certbot;
- a **one-shot migration service** that must succeed before the API and worker start;
- **health and readiness endpoints** checked by Compose and by the deployment workflow;
- the **continuous-delivery workflow** of Chapter 3, from the quality gates to the image registry and the server.

The domain `inkwell-ai.me` is registered. The remaining step — provisioning the server and pointing the domain at it — is listed with the perspectives.

Conclusion

Release 3 delivered the marketplace end to end — subscriptions and credits, the eligibility gate, the exclusive two-stage purchase, the magazine's own publication and the writer's itemised earnings, all on a ledger whose invariants are tested after every operation — together with moderation, a test suite of 792 backend tests, a demonstration corpus and a production-ready deployment. The last release deepens the social layer and reworks the assistant.

Chapter 7 — Release 4: social depth, the reworked assistant and documents

Introduction

The last release deepens what the platform already did. Sprint 7 completes the social layer with comment likes, follower lists, blocking, saving and sharing. Sprint 8 reworks the assistant: it now decides on each request whether to answer or to write into the document, shows its real pipeline as it runs, and accepts and speaks voice; the feed card becomes interactive. Sprint 9 gives the writer a private library of reference documents that the assistant can draw on and cite by page.

7.1 Sprint 7 — Comment likes, lists, block, save and share

7.1.1 Sprint goal and backlog

Goal: readers like comments, browse and act on follower lists, block accounts, keep articles for later and share them outside the platform.

Table 7.1 – Sprint 7 backlog

ID	User story	Priority	Points
US-57	Save an article to come back to it later, without anyone else seeing what I keep	Must	5
US-58	Pass an article on to people outside the platform so that they can read it without signing up	Should	3
US-56	Block an account so that I stop seeing someone without having to explain myself	Must	5
	3 stories		13

7.1.2 Analysis

Figure 7.1 gives the use cases of Sprint 7. Sharing and the follower lists need no account; following from a list extends viewing it. Liking a comment and saving an article include notifying the recipient. A block hides the two accounts from each other — profiles, articles and comments — and the only place to lift it is the blocker’s settings. Saves are private: the author is notified, but nobody else can see what a reader keeps.

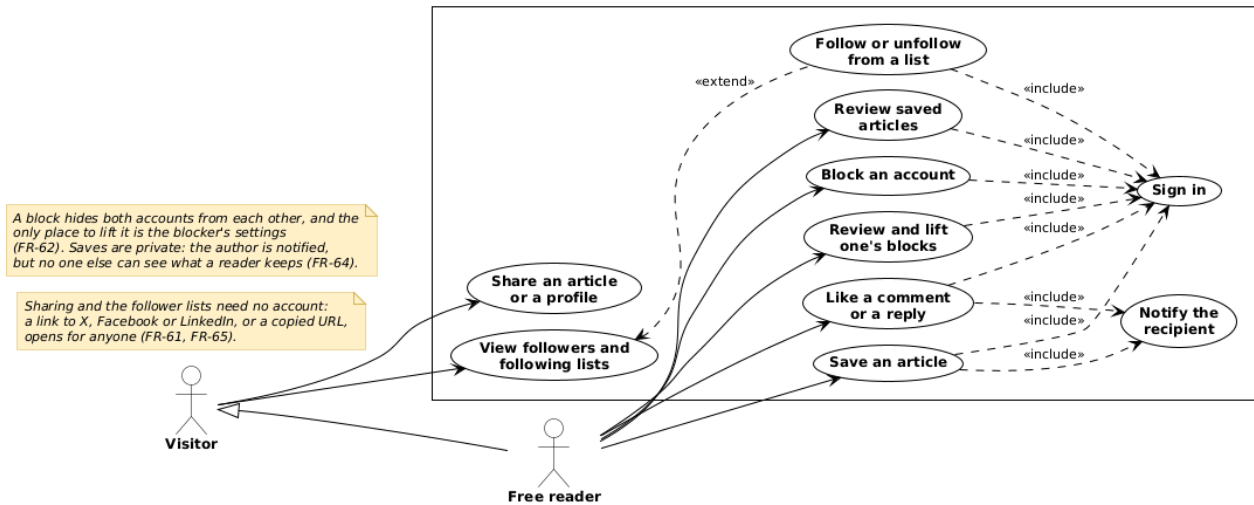


Figure 7.1 – Use case diagram, Sprint 7

7.1.3 Design

Figure 7.2 shows the three new tables — CommentLike, Block and Save — and the two notification types they emit. Each carries a unique constraint on its pair, so a repeated request is a no-op, and a notification is emitted only when a row was actually inserted; a check constraint forbids blocking oneself. A block row is directional, but every read treats it both ways. Comment likes are kept in their own table rather than among article likes, so that by construction they can never count towards a writer's eligibility.

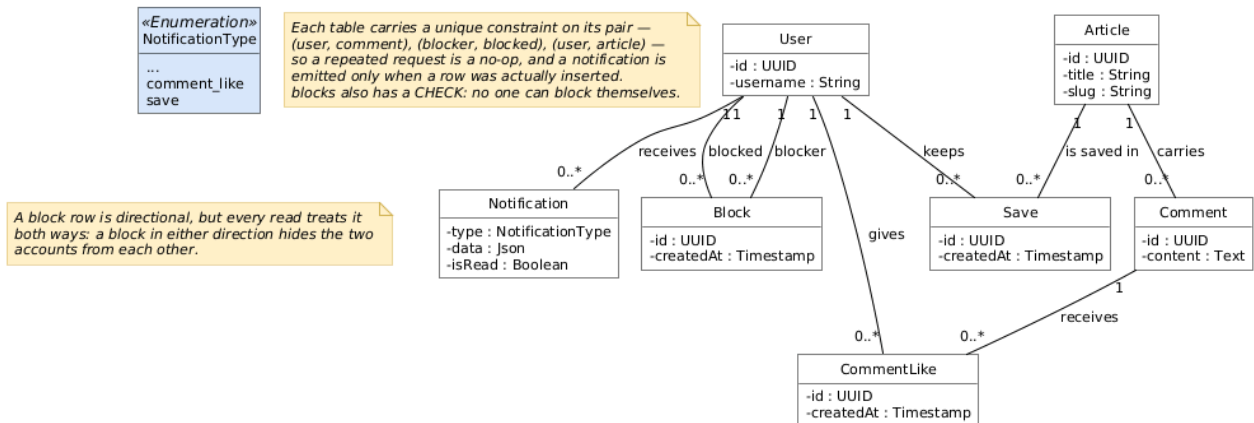


Figure 7.2 – Class diagram, Sprint 7

7.1.4 Realisation

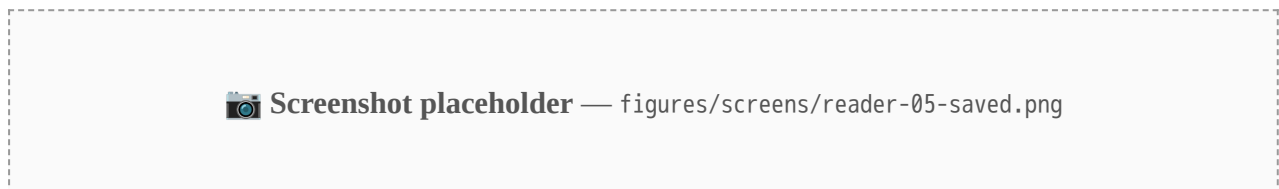


Figure 7.3 – The Saved shelf

 **Screenshot placeholder** — figures/screens/reader-06-blocked-accounts.png

Figure 7.4 – Blocked accounts, in settings

 **Screenshot placeholder** — figures/screens/reader-07-following-feed.png

Figure 7.5 – The following feed

 **Screenshot placeholder** — figures/screens/reader-08-followers-search.png

Figure 7.6 – A follower list, searched

7.2 Sprint 8 — Live cards, the assistant that writes, and voice

7.2.1 Sprint goal and backlog

Goal: readers react, repost and comment from the feed card, see recent comments and preview any byline; the assistant decides whether to answer or to write into the document, writes into a protected range the writer keeps or discards, shows its steps as they run, sits in a panel docked beside the editor, accepts dictated prompts and reads its replies aloud.

Table 7.2 – Sprint 8 backlog

ID	User story	Priority	Points
US-59	React to an article from the feed so that I can engage without losing my place	Must	5
US-60	See what an account has reposted so that I can judge what they rate	Should	3
US-61	Reply in the moment from the feed so that a fresh conversation is easy to join	Could	3

ID	User story	Priority	Points
US-62	Keep reading as I scroll so that a list does not end at an arbitrary point I have to click past	Must	5
US-65	See who a writer or magazine is from where their name appears so that I can decide to follow without leaving the feed	Should	3
US-66	Tell the assistant what to write and watch it appear in my draft, then keep or discard it, so that generation happens where I write rather than in a chat I copy from	Must	8
US-67	See what the assistant is doing while it works — and minimize it without losing sight of that — so that a slow step is not a frozen screen	Should	3
US-68	Say my request instead of typing it so that I can brief the assistant while my hands are busy or my thoughts are faster than my typing	Should	5
US-69	Hear the assistant's recap read out so that I can keep my eyes on the draft — and mute it when I would rather not	Could	3
	9 stories		38

7.2.2 Analysis

Figure 7.7 gives the use cases of Sprint 8. Briefing the assistant specialises into receiving an answer in the panel and having text written into the document, and includes following the live pipeline steps and consuming the allowance; keeping or discarding the written text includes the write. Dictating extends the brief and is billed; hearing the reply extends the answer. The choice between answering and writing is the assistant's, made per message; it supersedes the manual “insert the answer into the document” of Sprint 2 (FR-30, superseded by FR-78).

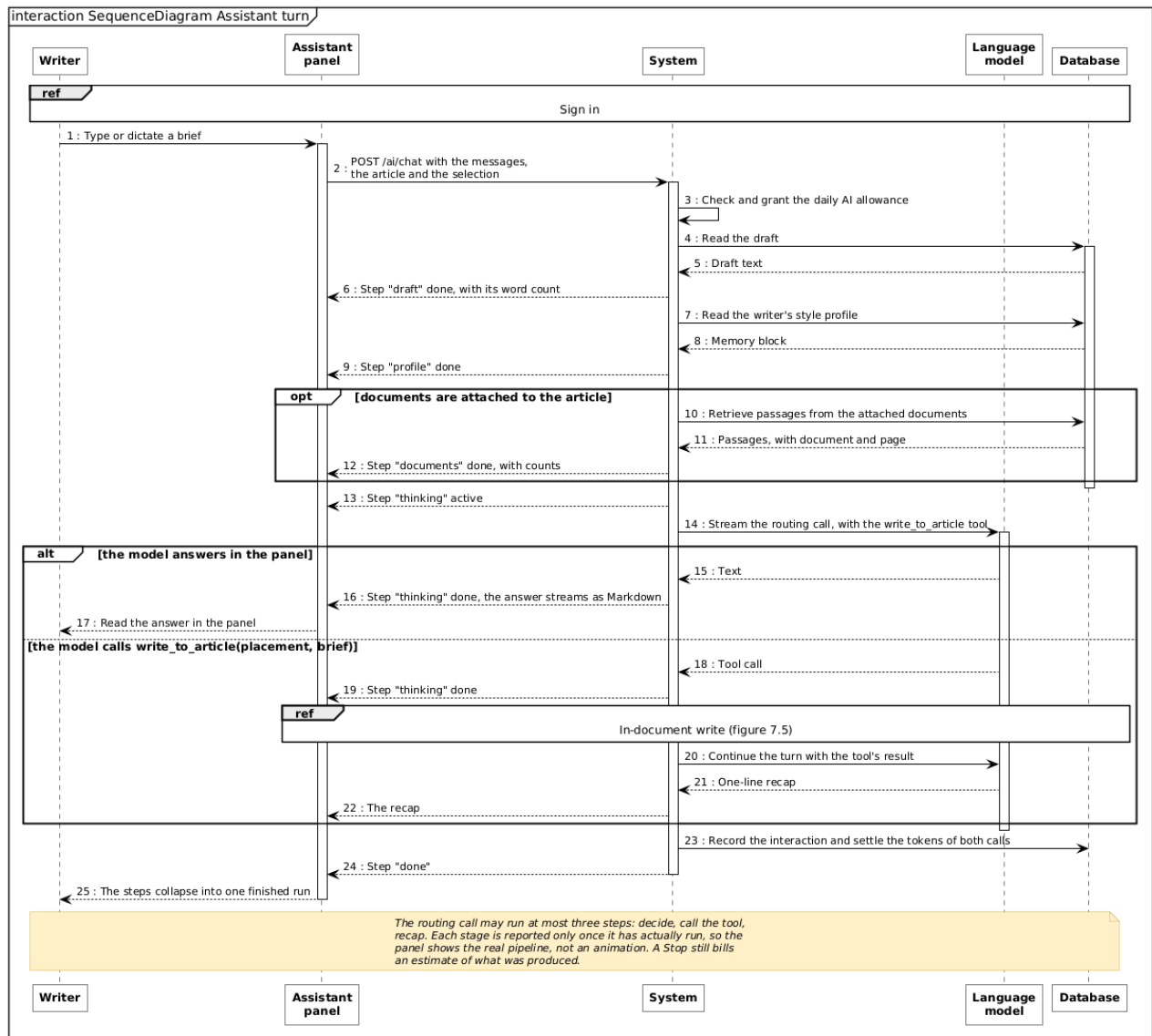


Figure 7.8 — Sequence diagram: one assistant turn

Sequence diagram — writing into the document. Figure 7.9 shows the write. The system retrieves passages from the writer’s other published articles and streams the article text from a second model call. The panel opens a range in the editor at a block boundary — after the block holding the cursor, or over the selected blocks — tints it and refuses the writer’s own edits inside it; each Markdown block is inserted as soon as it is complete, outside the undo history. When the stream ends, the writer keeps the text, which becomes one undo step, or discards it, which restores exactly what was there. Opening the range at a block boundary is what makes that restoration exact: every node the writer owns stays untouched.

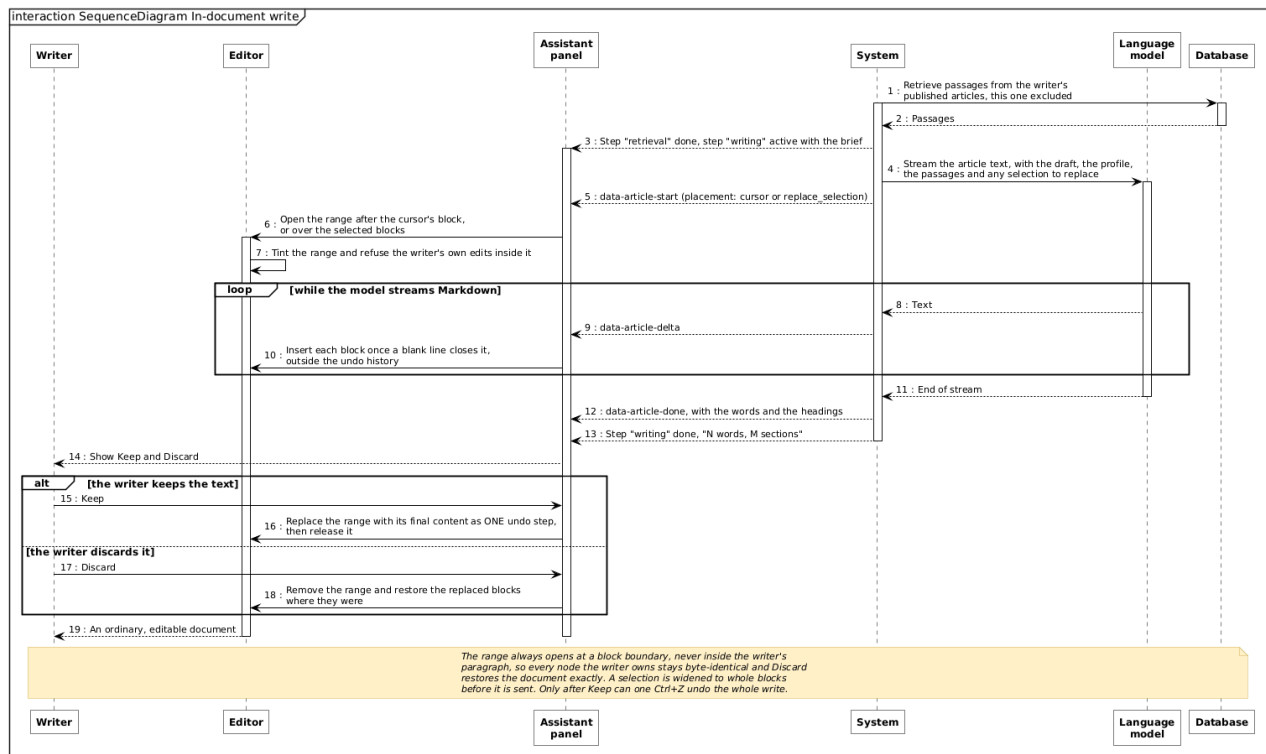


Figure 7.9 – Sequence diagram: writing into the document

Sequence diagram — voice. Figure 7.10 shows the voice round trip. A recording of at most five minutes is sent with its length; the system checks the allowance, the format and the size, transcribes it with Whisper, settles the duration — the provider's when it reports one, otherwise the larger of the client's figure and a floor derived from the file size — and bills 200 tokens per started minute. The transcript lands in the composer, editable. Reading a reply aloud is free to the writer and limited per account instead, to 20 replies a minute and 300 a day; the control exists only when speech synthesis is available, and the mute choice persists in the browser.

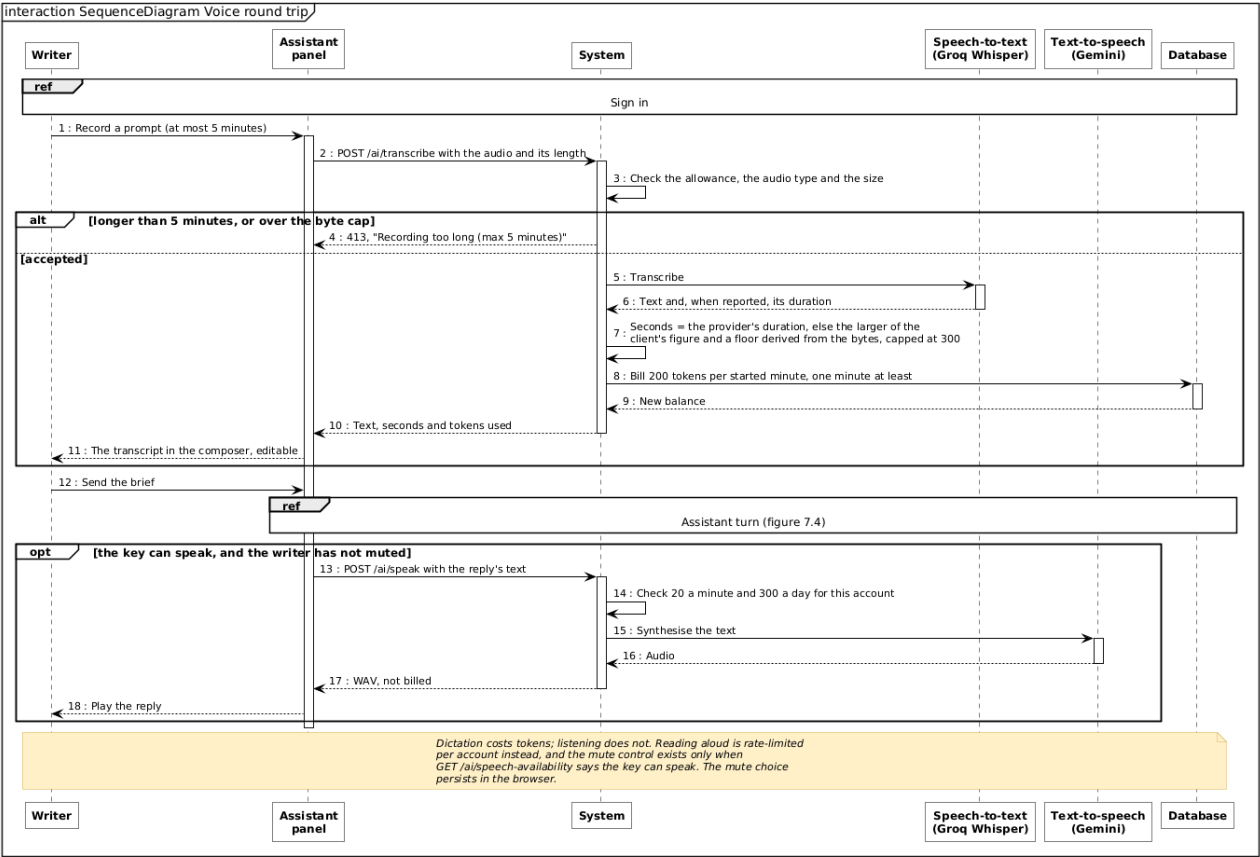


Figure 7.10 – Sequence diagram: the voice round trip

7.2.4 Realisation

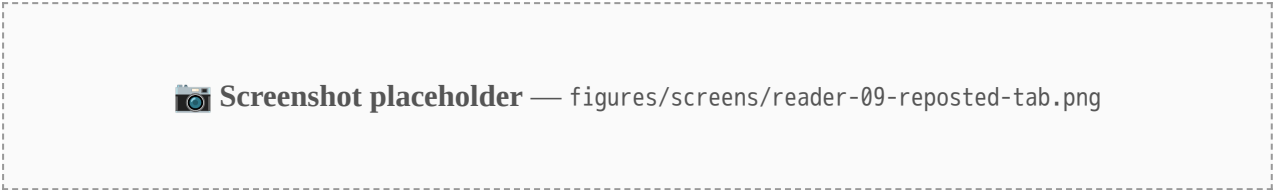


Figure 7.11 – The Reposted tab of a profile



Figure 7.12 – A write awaiting Keep or Discard

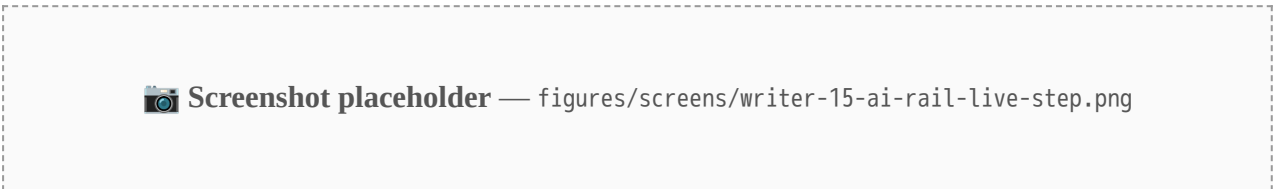


Figure 7.13 – The live step on the collapsed panel

 **Screenshot placeholder** — figures/screens/writer-19-voice-recording.png

Figure 7.14 – Recording a prompt

7.3 Sprint 9 — Document library and citations

7.3.1 Sprint goal and backlog

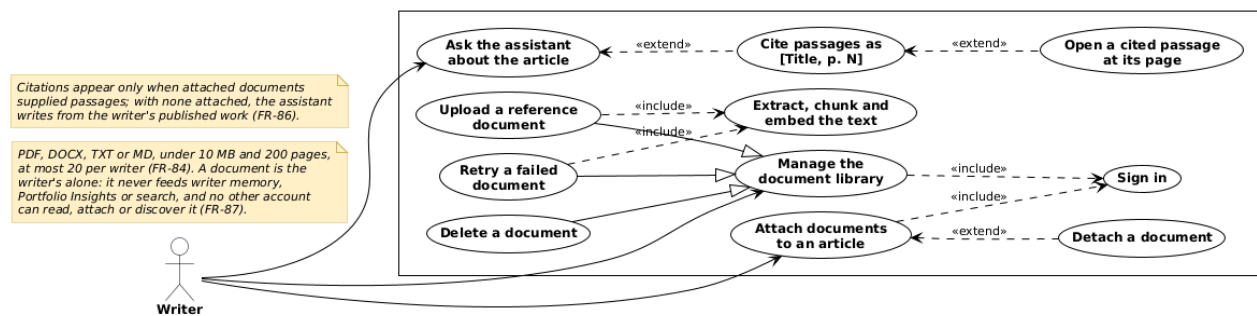
Goal: a writer uploads reference documents into a private library, attaches them to an article, and the assistant draws on them for answers and writes, citing each passage by document and page.

Table 7.3 – Sprint 9 backlog

ID	User story	Priority	Points
US-70	Upload the documents I am working from so that the assistant can use their facts without my retyping them	Must	8
US-71	Choose which documents apply to an article so that a piece on tides is not fed my notes on fermentation	Must	3
US-72	See which document and page a claim came from so that I can check it before it goes out under my name	Should	5
	3 stories		16

7.3.2 Analysis

Figure 7.15 gives the use cases of Sprint 9. Managing the library specialises into uploading, retrying a failed document and deleting; uploading and retrying include extracting, chunking and embedding the text. Attaching documents to an article is extended by detaching one. Asking the assistant is extended by citing passages as *[Title, p. N]*, itself extended by opening a cited passage at its page. Documents are PDF, DOCX, TXT or Markdown files under 10 MB and 200 pages, at most 20 per writer. A document is the writer's alone: it never feeds the voice profile, portfolio insights or search, and no other account can read, attach or discover it.



readable reason, then chunks it, embeds the chunks and marks it ready in one transaction. The file is served back only to its owner, through the API, as a short-lived URL.

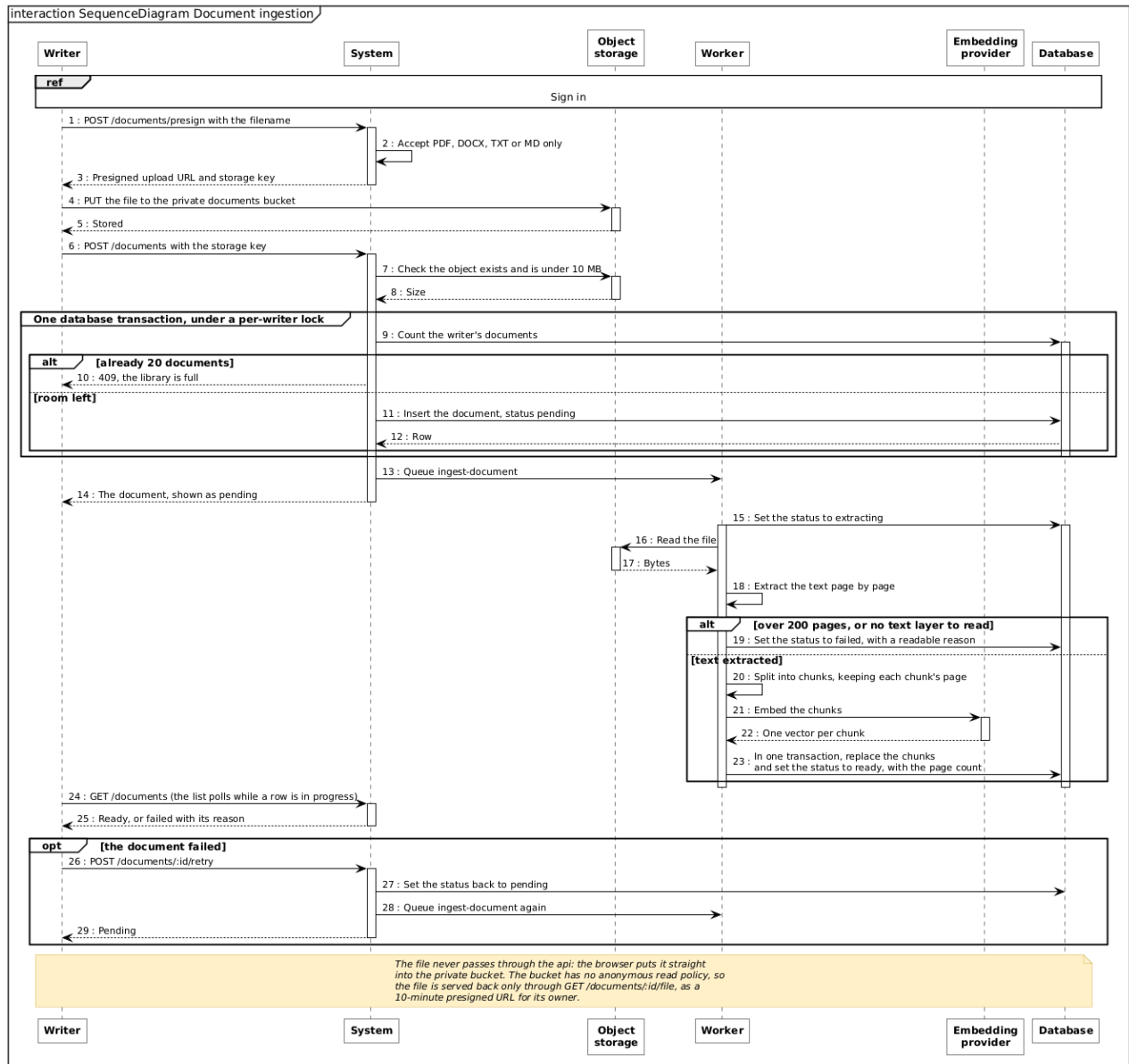


Figure 7.17 – Sequence diagram: document upload and ingestion

7.3.4 Realisation

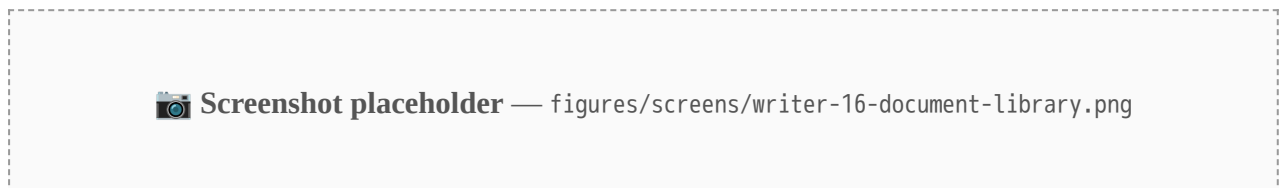


Figure 7.18 – The document library, one document refused with its reason

 **Screenshot placeholder** — figures/screens/writer-17-ai-sources-tab.png

Figure 7.19 – Attaching a document from the Sources tab

 **Screenshot placeholder** — figures/screens/writer-18-ai-cited-reply.png

Figure 7.20 – A reply citing its source as [Title, p. N]

Conclusion

Release 4 completed the social layer, turned the assistant from a chat beside the editor into a collaborator that writes into the document under the writer's control, added voice in both directions, and gave the writer a private library of sources the assistant can cite to the page.

General conclusion and perspectives

This project set out to connect two groups the existing platforms leave apart: independent writers, who lack an assistant that writes in their own voice and a direct way to be paid for quality, and magazines, which lack objective evidence to judge a writer before buying. Inkwell.ai answers both with one platform.

Over nine two-week sprints in four releases, the project delivered:

- a **writing environment** — a rich-text editor with automatic saving and images, and an AI assistant grounded in the writer’s own published work through retrieval-augmented generation, which decides per request whether to answer or to write into the document, writes into a protected range the writer keeps or discards, shows its real pipeline as it runs, accepts dictated prompts, reads its replies aloud, and cites reference documents to the page;
- an **analytics pipeline** — reading events captured in the browser, aggregated incrementally into per-article and per-writer metrics, presented to writers as dashboards and to magazines as an evaluation report completed by AI portfolio insights;
- a **marketplace** — an eligibility gate, magazine subscriptions with monthly credits, a two-stage exclusive purchase and the magazine’s own publication, with every credit movement written to a ledger inside one transaction and three balance invariants asserted after every money-moving test;
- a **social layer** — likes, threaded comments, comment likes, reposts, follows, saves, blocks, sharing and live notifications — and the **moderation** console;
- the **engineering around it** — a containerised stack started by one command, a continuous-integration pipeline with strict typing, zero lint warnings and a schema-drift check, 792 backend tests and 183 end-to-end tests, a demonstration corpus, and a production-ready deployment.

Beyond the product, the project was an exercise in engineering discipline: requirements stated per actor with stable identifiers, non-functional requirements tied to the mechanism that provides them, business rules proven by tests — including tests designed to fail when the code is sabotaged — and design documents and diagrams kept as versioned text beside the code.

Limitations

Some points remain open and are stated as such:

- **Live hosting.** The production stack, its TLS configuration and its delivery pipeline are ready and the domain is registered, but the server has not yet been provisioned.
- **Refresh-token revocation.** Refresh tokens are stateless; their 7-day lifetime is the bound on their validity, and a password change or a ban does not revoke one already issued.
- **Pagination.** Lists are paginated by offset; a cursor-based scheme would stay stable when new rows are inserted while a reader scrolls.
- **Accessibility and performance audit.** A Lighthouse score above 90 and the absence of critical accessibility violations on article pages (NFR-37) remain to be measured.

- **Readiness.** The readiness endpoint checks PostgreSQL but not Redis.

Perspectives

- **Go live** on a provisioned server, then measure real traffic against the performance requirements.
- **Real payments** in place of the simulated ones, with the ledger already designed to record them.
- **Revocable sessions**, by storing refresh tokens server-side.
- **Richer AI collaboration:** dictating whole articles, writer-specific style learning beyond the current voice profile, and feedback on weak sections.
- **Wider audience:** multilingual publishing and a native mobile application on the existing 375 px designs.
- **Deeper analytics:** geographic and device distribution collected with the reader's consent, and topic-relevance ordering when magazines browse writers.

References

1. Schwaber, K. and Sutherland, J. *The Scrum Guide*. scrumguides.org, 2020.
2. Object Management Group. *Unified Modeling Language (UML)*, version 2.5.1. 2017.
3. Lewis, P. et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. *Advances in Neural Information Processing Systems* 33, 2020.
4. Cormack, G. V., Clarke, C. L. A. and Büttcher, S. “Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods”. *Proceedings of SIGIR*, 2009.
5. Malkov, Y. A. and Yashunin, D. A. “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs”. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2018.
6. Next.js documentation — nextjs.org/docs.
7. NestJS documentation — docs.nestjs.com.
8. PostgreSQL 16 documentation — postgresql.org/docs/16 (full-text search).
9. pgvector — github.com/pgvector/pgvector.
10. BullMQ documentation — docs.bullmq.io.
11. Drizzle ORM documentation — orm.drizzle.team.
12. Vercel AI SDK documentation — ai-sdk.dev.
13. TipTap documentation — tiptap.dev.
14. Docker Compose documentation — docs.docker.com/compose.
15. Playwright documentation — playwright.dev.